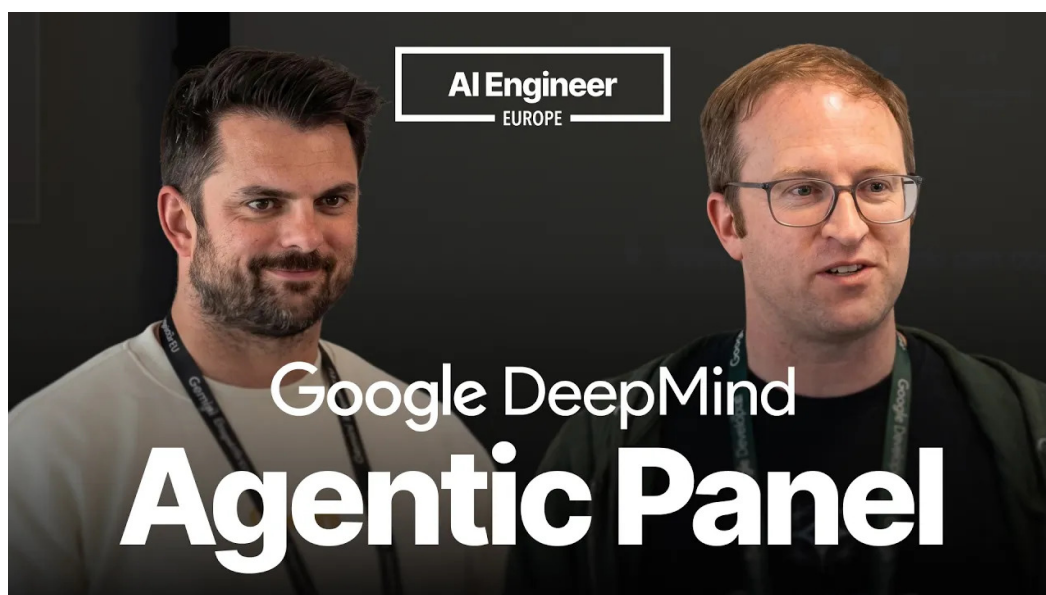


视频课程笔记

How Google DeepMind Runs Agents at Scale —KP Sawhney & Ian Ballantyne, Google DeepMind

五道口纳什 & Codex

2026 年 5 月 31 日



视频频道：AI Engineer

发布日期：2026-05-24

视频时长：25:12

视频链接：<https://www.youtube.com/watch?v=7gujZrJ9L5I>

目录

1 从演示开始: DeepMind 为什么把智能体当作平台问题	3
1.1 两位讲者与议题边界	3
1.2 从单次对话到可运营的智能体系统	4
1.3 本章小结	4
2 Antigravity 演示: IDE 里的智能体管理器	5
2.1 Agent Manager: 同时运行多个任务代理	5
2.2 计划、工具调用与浏览器控制	6
2.3 DOM 检查、截图视频与人工确认	7
2.4 本章小结	8
3 Deep Research Agent: 从研究流水线到共享工作空间	8
3.1 Deep Research 的现有管线约束	9
3.2 为什么共享文件系统能降低上下文搬运	10
3.3 本章小结	11
4 Skills Library: 组织知识如何进入智能体	11
4.1 技能是专家经验的可复用封装	11
4.2 Agent Sprawl: 规模组织里的能力失控风险	12
4.3 技能筛选与“达尔文式”治理	13
4.4 本章小结	14
5 Google 规模下的硬约束: Token、Quota 与计算资源	14
5.1 为什么智能体工作流特别吃 Token	15
5.2 按用户与团队管理配额	16
5.3 在本地模型、便宜模型与强模型之间分工	16
5.4 Mock TPU 与低成本评测环境	17
5.5 本章小结	18
6 Per-user Behavior: 一个强用户怎样拖垮系统	18
6.1 多实例、多代理与突发负载	18
6.2 粗粒度限额的现实做法	19
6.3 订阅制遇到智能体工作流的张力	20
6.4 SRE 视角: 什么时候算太多	20
6.5 本章小结	20
7 Observability: 看见智能体轨迹, 才能调试智能体	21
7.1 统一后端与层级化排查界面	21
7.2 Trajectory Store: 保存步骤、循环与失控点	22
7.3 从 Predict Request 到工作流诊断	22
7.4 本章小结	23
8 Deep Research 的下一步: 让管线成员像研究同事一样协作	23
8.1 上下文窗口如何被长研究任务消耗	23
8.2 工作空间协作模型	24
8.3 信息图、支撑材料与文档产物	25
8.4 本章小结	25
9 Multi-agent System: 多轨并行与人工监督	25
9.1 Antigravity 中的多代理呈现方式	26
9.2 Agent-to-agent Communication 的效率问题	26
9.3 人类像装配线主管一样塑造协作	26

9.4 本章小结	27
10 Skills 与 MCP：能力接入的两条路线	27
10.1 Skills：把专家知识交给代理	28
10.2 MCP：认证与外部工具协议的价值	29
10.3 Guardrailed CLI：在受控命令行中完成任务	30
10.4 本章小结	30
11 Evaluation：复杂智能体 workflow 怎样被验证	30
11.1 沙箱环境搭建的机械成本	30
11.2 新数据集为什么最难	31
11.3 技能作者、代理生成测试与元评测	32
11.4 本章小结	32
12 Quota Management：模型限制不能中断任务流	32
12.1 企业账户、个人账户与不同层级限制	33
12.2 从强模型降级到 Flash 或本地模型	33
12.3 把配额耗尽变成可预期状态	34
12.4 本章小结	35
13 Automated Code Review：当代理写更多代码，审查也要升级	35
13.1 按语言微调的自动审查模型	35
13.2 按产品线配置的 SRI 与 Prompt	36
13.3 PR 里的代理评论与审查规模膨胀	36
13.4 本章小结	37
14 总结与延展：智能体平台的五条控制线	37
14.1 全片主线压缩：从演示闭环到生产控制	37
14.2 五类失败模式与对应控制	38
14.3 对研究、产品和工程团队的具体启示	39
14.4 开放问题：当平台继续放大	40
14.5 本章小结	40

1 从演示开始：DeepMind 为什么把智能体当作平台问题

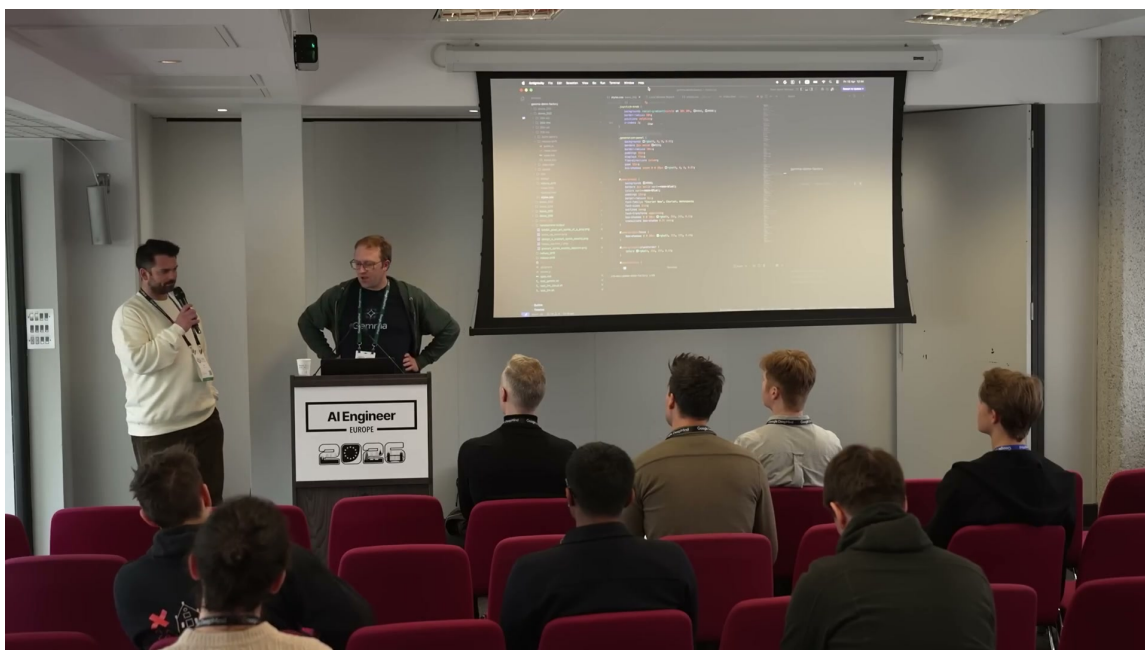


图 1: 开场段落中的两位讲者与 AI Engineer Europe 会场背景；该画面用于交代视频来源、讨论场景和现场问答属性。¹

1.1 两位讲者与议题边界

这场讨论由 Google DeepMind 的 Ian Ballantyne 与 KP Sawhney 开场。Ian 的身份是 Google DeepMind 的开发者关系工程师，KP 来自 DeepMind 的 AI Platform 团队。这个组合本身就给出了整场分享的边界：一边面向开发者理解工具如何被使用，一边面向平台团队理解这些工具怎样被组织、扩展、约束和运行。

开场不到一分钟，讲者没有先堆模型参数、提示词模板或 benchmark 数字，直接说要谈 DeepMind 如何思考 agentic software，如何构建自己的技术栈，并马上进入一个演示。这个顺序很重要。对智能体系统而言，真正难的部分往往出现在模型调用之后：任务怎样被拆解，工具怎样被授权，浏览器和 IDE 状态怎样被观察，失败怎样被暴露，用户怎样在关键节点介入。

本章先固定的 agent 定义

在这份笔记中，**agent（智能体）**指一个由模型驱动的工作单元：它接收目标，规划步骤，调用工具，观察结果，并产出可审查的工件或状态变化。这个定义比“会聊天的模型”更窄，也比“完全自主的软件员工”更克制。它强调的是可执行的闭环：目标进入系统，行动改变环境，观察结果回到系统，人类可以检查过程和产物。

这个定义有几个隐含条件。第一，智能体需要目标；没有目标，模型只是在续写文本。第二，智能体需要工具；没有工具，它只能生成建议，无法真正修改代码、打开浏览器或检查页面。第三，智能体需要观察；没有观察，它无法知道行动是否真的改变了外部世界。第四，智能体需要可审查产物；没有审查面，人类只能相信结果，无法判断过程是否可靠。

¹ 视频画面时间区间：00:00:18–00:00:35。

1.2 从单次对话到可运营的智能体系统

单次 prompt 的价值在于表达意图，例如“根据这个 spec 写一个示例”。平台问题的价值在于把意图变成一条可以运行、可以中断、可以复查、可以复用的轨道。前者像给同事发一条消息；后者像搭建一条小型生产线：有任务入口，有工位，有检查点，有日志，有主管放行。

在智能体场景里，这个差别会迅速放大。一次提示词失败，用户可以重问；一个智能体平台失败，可能表现为错误文件被改写、浏览器状态无法复现、模型在循环里耗尽配额、用户看不到它为什么做出某个改动。讲者选择用演示开场，实际是在把注意力从“模型会不会回答”移动到“系统能不能承载连续行动”。

“agentic software”的工程含义

这里的 agentic software 可以理解为一类带有行动循环的软件：它把模型输出接到工具、文件系统、浏览器、规划器和审核界面上。类比传统后端服务，模型像决策函数，工具像外部 API，观察结果像返回值，轨迹记录像日志，人类审批像发布闸门。缺少这些外层结构，模型能力很难稳定变成工程生产力。

因此，DeepMind 把智能体放到平台视角里讨论，有一个很现实的原因：当使用者变多、任务变长、工具变杂时，系统的瓶颈会从“模型是否聪明”扩展到“运行时是否可控”。一个能写代码的模型，只解决了能力来源；一个能管理任务、工具、上下文、证据、审批和轨迹的平台，才开始触及生产环境。

常见误解：把智能体等同于更长的 prompt

更长的 prompt 只能给模型更多文字背景，不能自动提供工具权限、浏览器控制、DOM 观察、失败恢复或人工审批。把智能体理解成“更长、更复杂的提示词”，会漏掉平台层真正消耗工程成本的部分：状态管理、权限边界、可观测性、配额和审查。

1.3 本章小结

第一章的核心作用是确定阅读角度。Ian Ballantyne 与 KP Sawhney 讨论的主题，重点在于 Google DeepMind 如何把智能体放进可运行的平台栈里，孤立提示词的写法只占很小一部分。本文后续使用的 agent 定义，也会保持这个边界：它是接收目标、规划、行动、观察、产出可审查结果的模型驱动工作单元。下一章的 Antigravity 演示，就是这个定义落到 IDE 和浏览器里的具体样子。

2 Antigravity 演示：IDE 里的智能体管理器

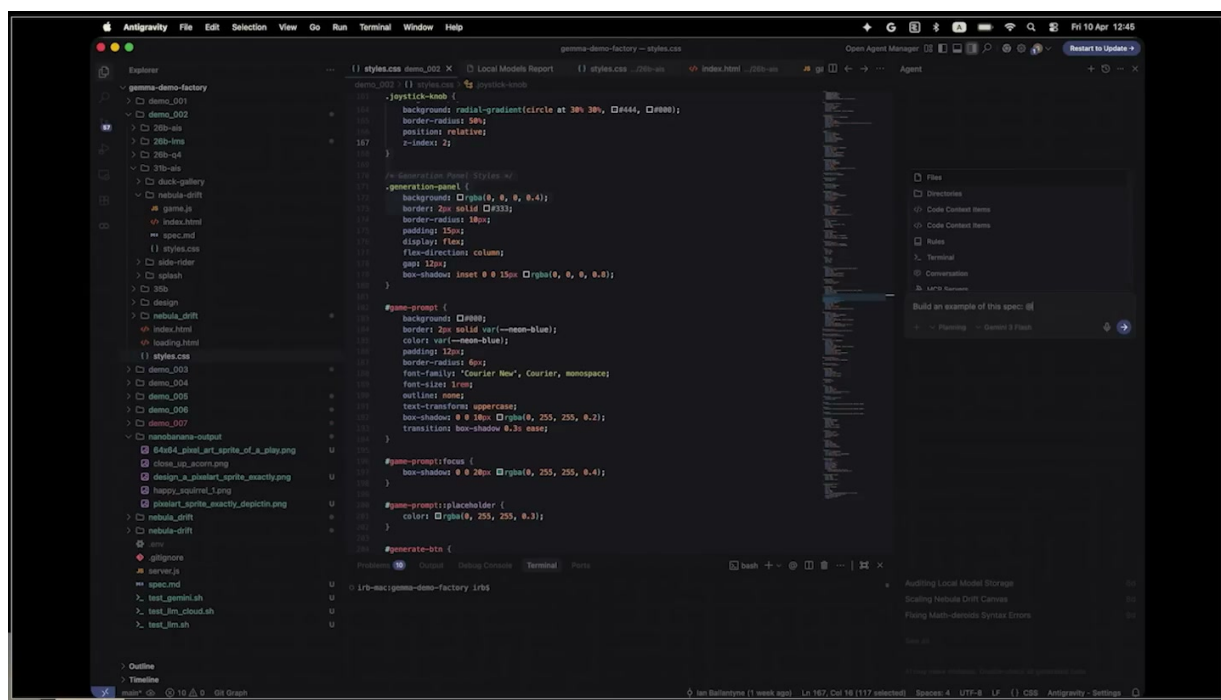


图 2: Antigravity IDE 中的项目文件树、Agent Manager 面板与 spec 输入入口，展示任务如何从工程上下文进入代理管理器。²

2.1 Agent Manager：同时运行多个任务代理

演示从一个现场提问开始：有多少人已经使用过 Antigravity？台下只有少数人举手。讲者随后点出一个容易被忽略的事实：很多人知道 Antigravity 有类似 Visual Studio 的界面，却没有意识到它背后有完整的 agent manager 和 agent manager framework。这个说法把 Antigravity 从“带聊天栏的 IDE”提升为一个智能体运行环境。

Agent Manager 的关键能力，是让用户在同一个 IDE 语境中启动、生成和管理多个智能体，让它们处理不同项目或不同任务轨道。这里的“管理”不能只理解成进程列表。它包含任务入口、上下文绑定、模型选择、工具调度、计划显示、浏览器控制、执行报告和用户反馈入口。用户看到的是侧边栏里的聊天和任务状态，平台实际承担的是一组控制面。

agentic platform 与 Antigravity 的定义

agentic platform (智能体平台) 是围绕智能体构建的运行与治理外壳，包括任务路由、工具访问、规划系统、浏览器与 IDE 控制、轨迹捕获、配额策略和人工审批。

Antigravity 平台是讲者演示的 IDE 式智能体平台：它把 Agent Manager、chat/spec 输入、模型规划、内部工具、浏览器控制、DOM 检查、执行证据、计划审查和人工确认放进同一个开发 workflow。产品名保留英文 Antigravity，避免把它误读成普通物理概念。

演示中的入口很普通：用户在侧边 chat panel 里输入“build an example of this spec”，并指定一个 spec 文件。普通聊天工具到这里通常就结束了：模型生成一段建议或代码。Antigravity 的差异在于，输入被转成一个可跟踪任务。模型在侧边开始“thinking”，读取 spec，分析已有文件，然后调用内部

²视频画面时间区间：00:01:31–00:01:48。

工具决定下一步。字幕里提到它发现已有文件，甚至可能判断任务已经完成，这说明平台把文件系统状态纳入了观察范围。

现场还有一个值得记录的小插曲：最初发送给 Flash 时出现错误，讲者重启后继续演示。这个片段没有教学上的戏剧性，却有平台工程上的真实性。智能体平台依赖模型服务、网络、IDE、本地项目和浏览器，多组件链路越长，越需要清楚的失败提示和恢复路径。平台问题比单次 prompt 更关键，正是因为它要处理这类真实运行摩擦。

2.2 计划、工具调用与浏览器控制

Antigravity 随后展示出两个核心内部结构：built-in to-dos 和 planning system。To-do 列表让任务从一句自然语言变成若干可执行小步；planning system 则决定这些小步如何排列、哪些文件需要检查、何时打开浏览器验证结果。可以把它想成自动驾驶里的导航系统：模型提供判断，规划器把判断落实到路线，工具调用像方向盘和油门，观察结果像传感器回读。

为了描述这个闭环，可以先用一句中文压缩它：智能体平台每一步都在“根据目标和当前状态制定计划，执行工具动作，观察外部世界，再把观察交给人类或下一步计划”。这个循环可以写成一个简化公式：

$$x_{t+1} = H(\text{Observe}(\text{Act}(\text{Plan}(g, x_t, \mathcal{T}))))$$

其中：

- g 表示用户给出的目标，例如“根据这个 spec 构建示例”。
- x_t 表示第 t 步时平台可见的状态，包括文件、浏览器、任务记录和已有分析。
- $\mathcal{T}$ 表示平台允许智能体调用的工具集合，例如文件读取、编辑、浏览器控制和 DOM 检查。
- Plan 表示把目标和状态整理成可执行计划。
- Act 表示按计划调用工具并改变环境。
- Observe 表示读取行动后的结果，例如页面是否启动、DOM 是否符合预期、文件是否已改动。
- H 表示人类把关函数：人类可以查看、修改、批准、打断或重定向这一轮结果。
- x_{t+1} 表示经过行动、观察和人工把关后的新状态。

这个公式只是教学抽象，并非讲者给出的数学模型。它的价值在于把“智能体在工作”拆开：计划、行动、观察、审批四件事必须接上，闭环才完整。缺少计划，系统只是在随机尝试；缺少工具，计划无法落地；缺少观察，行动无法校验；缺少人工把关，错误可能被自动放大。

演示中，Antigravity 打开浏览器来运行应用，并且这个浏览器可以由 Antigravity 自身控制。这里的浏览器不只是预览窗口，它是外部世界的一部分。对于 Web 项目，代码是否正确，不能只看文件 diff，还要看页面是否真的启动、交互是否可用、状态是否符合 spec。平台把浏览器接进闭环，等于让智能体从“写代码”跨到“运行并检查代码”。

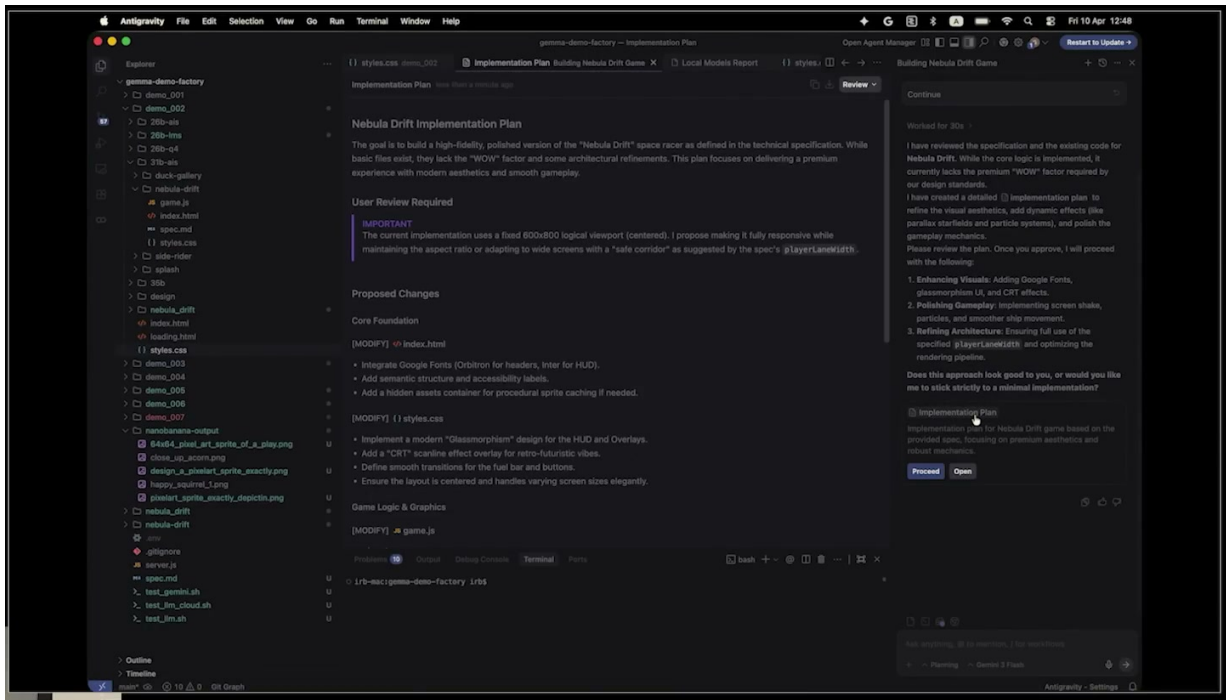


图 3: Antigravity 生成实施计划后要求人工审阅；右侧对话面板给出继续执行前的确认入口，体现人在回路中的控制点。³

2.3 DOM 检查、截图视频与人工确认

演示继续向下走，讲者指出 Antigravity 可以 inspect the DOM，也就是检查网页实际的文档对象模型。DOM 是浏览器把 HTML 页面解析成的树形结构，按钮、输入框、文本区域、列表项都以节点形式存在。智能体能看 DOM，就像测试工程师能打开开发者工具看页面骨架；它不必只依赖截图里的像素，还可以读取页面结构和元素状态。

DOM inspection 为什么比“看截图”更硬

截图告诉系统“页面长什么样”，DOM 检查告诉系统“页面由什么结构组成”。前者像看一张店铺照片，后者像拿到货架清单和商品位置。对智能体而言，DOM 可以帮助定位按钮、文本、错误区域和交互状态；截图和视频则提供视觉证据，适合给人类复核交互过程。两者结合，才更接近可审查的前端验证。

讲者还提到，任务完成后平台会给出 report，说明它完成了什么、实现了什么，用户可以据此 review。它还可以捕获 screenshot 或 video，尤其适合“给网页加一个功能”这类交互任务。这里的截图和视频承担证据功能。代码 diff 说明“文件变了”，DOM 检查说明“结构可见”，截图或视频说明“用户路径被实际跑过”。三者一起构成更强的审查面。

演示中还有一个很能说明平台边界的细节：页面边缘出现蓝色边框时，讲者说明这是 Antigravity 正在接管浏览器里的游戏。它先尝试启动游戏，不确定控制方式时可能继续查找和探索；随后讲者展示被编辑的文件，并指出原文件其实由另一个模型生成，当前结果已经根据 spec 被重写成了一个不同的游戏。这个片段说明 Antigravity 面对的是一个已有工程状态：旧文件、旧模型产物、浏览器运行结果、DOM 状态和人类审查共同构成任务环境，远比空白代码生成场景更复杂。智能体平台的难点，也正是

³ 视频画面时间区间：00:04:04–00:04:22。

在这些状态之间保持可观测、可回退、可审查。

随后出现 implementation plan。平台展示它认为应该修改文件的哪些地方，用户可以直接编辑某一行，告诉它“我想要不同的行为”，也可以指出“这完全偏离了我的意思”。这就是 human in the loop。人类在回路中的含义，是把判断力放到最有价值的位置：计划提交前、敏感修改前、结果验收前。

人在回路中、scratchpad 与控制闭环

人在回路中 (human in the loop) 是一种工作流模式：智能体提出计划或变更，人类可以编辑、批准、打断或重定向。

scratchpad (草稿轨迹/临时推理记录) 是演示中可见的智能体工作笔记，用来展示它在完成任务时记录的中间想法、观察和尝试。这里不能承诺每个平台都会暴露全部内部推理；本章只把它描述为 Antigravity 演示中可见的轨迹工件。

这两个机制合在一起，把“自动执行”变成“受监督的连续执行”：智能体可以推进任务，人类仍保留方向盘和刹车。

演示后半段还显示了 scratchpad：智能体写下自己在任务中尝试理解的行为，用户可以看到它正在寻找什么、如何判断页面、是否在尝试找控制方式。这个细节容易被低估。没有 scratchpad 或类似轨迹，用户只能看最终改动；有了轨迹，用户能判断错误源头是在 spec 理解、文件分析、DOM 观察、交互尝试，还是计划本身。

别把“自动化”理解成“放弃控制”

智能体平台的成熟方向不是让用户完全退出现场。更实际的方向是把人类从重复操作中移开，同时把审批、纠偏和验收位置做得更清楚。Antigravity 演示里的 proceed 按钮、计划编辑、报告、截图/视频和 scratchpad，都是控制权重新分配的证据。

把 Section 2 的演示连起来看，控制闭环大致是：用户在 chat/spec 中给目标和文件；Agent Manager 启动任务；模型读取 spec 和项目状态；规划系统生成 to-do 与 implementation plan；内部工具检查文件和现有实现；浏览器被打开并受平台控制；DOM inspection、截图和视频形成观察证据；scratchpad 和报告留下轨迹；人类审查计划，必要时修改，然后点击 proceed。这个链条解释了为什么 Antigravity 被称为 agentic platform：它提供的是一整套任务运行和审查结构。

2.4 本章小结

Antigravity 演示把智能体平台的核心部件摆在一个 IDE 工作流里：Agent Manager 负责多任务入口，chat/spec 负责表达目标，planning 和 to-do 负责把目标变成步骤，internal tools 负责访问项目状态，browser control 与 DOM inspection 负责观察真实应用，screenshot/video 负责提供复核证据，implementation plan 负责把将要发生的变化显性化，human approval 负责最后的方向控制。后续章节会把同一套思想推广到研究流水线、技能库、配额、可观测性和代码审查；本章给出的定义会作为后文的公共词汇。

3 Deep Research Agent：从研究流水线到共享工作空间

上一章的 Antigravity 演示把读者带进一个具体的智能体平台：用户给出目标，系统生成计划，调用浏览器和 DOM 检查工具，留下 scratchpad 与报告，再交给人类审阅。KP 随后把这个模式从编

码任务推进到研究任务。他提到自己几个月前参与了 Deep Research agent，并说明它已经可以通过 interactions API 使用；当前重点转向如何让 Antigravity 这类 harness 在内部承担更多类型的任务。

KP 先把这个 harness 的内部价值落到编码规模上：Google/DeepMind 有巨大的 mono repo，内部代码工作本身已经足够复杂，Antigravity 这类 harness 首先要服务大规模编码、调试和验证。随后他才把问题推广到 coding 之外的用例：如果同一套 harness 可以承载代码任务，它也可能承载研究、文档、信息整理等更长链路任务。这样理解，Deep Research 的话题切换来自同一条工程主线：复杂代码工作需要外部化状态，复杂研究工作也需要外部化状态。

3.1 Deep Research 的现有管线约束

本节里的 Deep Research 智能体，指一种面向研究工作的智能体或管线：它接收一个开放式研究目标，发起搜索，阅读材料，抽取证据，组织中间结论，最终产出报告或其他可审阅的研究成果。它和普通问答的区别在于任务跨度更长，材料来源更多，中间状态更重。普通问答像在白板上回答一道题；Deep Research 更像一个研究助理在桌面上摊开论文、网页、笔记、引用清单和草稿，然后逐步把材料压成结论。

术语定义：Deep Research 智能体

Deep Research 智能体是一类研究导向的智能体工作流：它围绕一个研究目标执行搜索、阅读、整理、综合和报告生成。它的核心负担不只在“生成最后一段文字”，还在管理搜索结果、证据片段、草稿版本、引用关系和未完成问题这些中间资产。

KP 点出的痛点非常具体：当前 Deep Research 这类系统在不同管线环节之间，会传递来自搜索的大块文本。搜索模块拿到网页和文档，摘要模块接收一批文本，综合模块再接收摘要、证据和历史判断。每一步都要把前一步的材料重新塞进模型调用里。这样做能跑起来，却像几个人合作写报告时每次都把整摞资料重新复印一遍，再塞进对方的信封。短任务还能忍，长研究会很快变得笨重。

可以把这种搬运成本写成一个简单的直觉式表达：

$$C_{\text{move}} = \sum_{k=1}^n |T_k| \cdot r_k$$

其中：

- C_{move} ：跨管线搬运研究文本的总负担，可以理解为 token、延迟和整理复杂度的合成成本；
- k ：第 k 类中间材料，例如搜索结果、摘要、证据表、草稿；
- $|T_k|$ ：这类材料的文本规模；
- r_k ：这类材料被后续环节重复传递、重读或重写的次数。

这个公式属于本节补充的教学辅助，用来帮助读者看清结构：成本的增长来自两件事，一是材料本身变大，二是材料被反复搬运。研究任务天然会积累材料，所以真正危险的是重复搬运。每个管线阶段都把“我需要知道的一切”塞进一次模型调用，结果会把研究系统变成一个不断复制文本的流水线。

这里要轻轻点到上下文窗口，但不展开定义。模型一次能直接注意到的内容有限，长研究任务会把这块空间消耗在历史材料、搜索结果和工具输出上。第 8 节会专门讨论这个压力；本节只抓住一个工程判断：当中间材料已经形成稳定文件或结构化记录时，继续把它们以长文本形式在阶段之间来回传递，效率很差，审查也很差。

3.2 为什么共享文件系统能降低上下文搬运

KP 提出的替代方向是让 Deep Research 管线的不同部分在共享文件系统里协作。这个想法的价值在于改变“信息交接”的形态。流水线传文本，像把整本笔记通过聊天框转发给下一位同事；共享工作空间则像几位研究同事围着同一个项目文件夹工作：有人维护资料库，有人写证据表，有人整理草稿，有人标注不确定点。协作者不必每次重述全部历史，只需要打开当前任务相关的文件。

共享文件系统的关键并不只是“把东西存下来”。存储只是第一层，真正的机制包括四件事。第一，材料有稳定地址，例如 `sources/search_results.md`、`notes/evidence_table.md`、`drafts/report_v2.md`。第二，中间产物可以被局部更新，而非整体重写。第三，不同管线角色可以围绕同一份文件形成读写约定。第四，人类可以审查这些文件，判断智能体到底依据了哪些材料、放弃了哪些线索、改写了哪些结论。

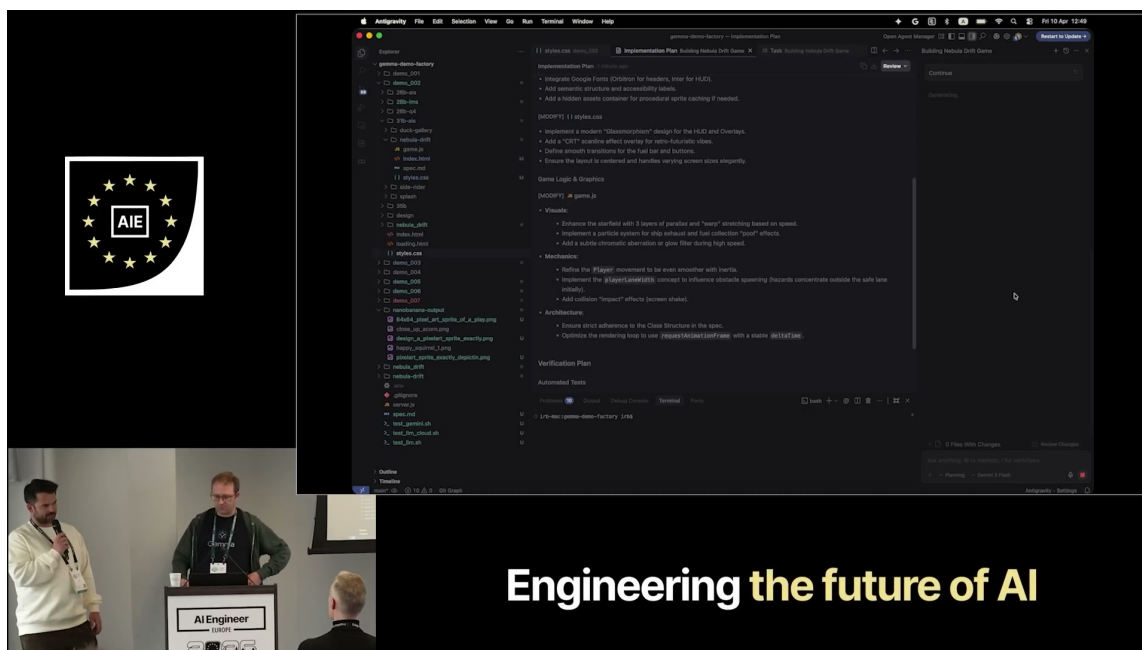


图 4: 讲者讨论 Deep Research 与共享工作空间时的代表画面；视频此处没有独立 Deep Research UI 或架构图，后文用生成示意图补足机制表达。⁴

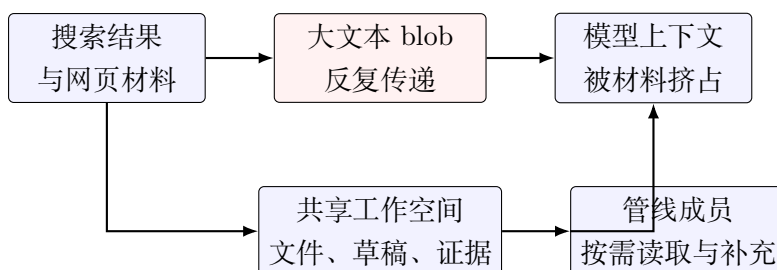


图 5: 根据 00:04:44–00:05:35 关于 Deep Research 管线的讨论重绘的说明性示意图：共享工作空间把材料留在可协作的外部状态中，减少长文本在上下文窗口中的重复搬运。

⁴视频画面时间区间：00:04:44–00:05:35。

共享工作空间的类比

传统管线像传送带：每个工位接收上一工位打包好的材料，再把自己的输出继续打包给下一工位。共享工作空间像项目室：资料、白板、草稿和任务清单都放在同一个房间里，不同成员按权限读取和修改。研究任务更适合后者，因为研究过程会反复回看材料、修正判断、补充证据，而非单向加工一次就结束。

这也解释了为什么 Antigravity harness 对 Deep Research 有启发。编码任务里，智能体可以读文件、改文件、运行浏览器、检查 DOM、留下计划和 scratchpad。研究任务也需要类似的“外部化工作台”：搜索结果应当成为可命名、可追踪、可引用的材料，避免长期停留在一次提示词里的临时文本团；分析步骤也应落到证据表、假设清单和报告草稿里，避免随一次回复结束而消失。

从组织工程角度看，共享工作空间还降低了审查成本。人类审阅一篇 Deep Research 报告时，最怕看到一个漂亮结论却找不到支撑材料。若每个阶段都把产物写进共享目录，审阅者就能追溯：哪些搜索命中了，哪些材料被纳入，哪些证据支持了哪个段落，哪些疑点仍未解决。研究智能体因此更像同事，少像一次性文本生成器。

3.3 本章小结

本章把 Deep Research 智能体定义为研究导向的长程智能体工作流。它的难点在于中间资产重：搜索结果、摘要、证据、引用、草稿和未决问题都会积累。KP 点出的核心问题是，大量文本在管线阶段之间被反复搬运，导致成本、延迟、上下文压力和审查难度一起上升。

共享文件系统或共享工作空间的价值，是把“传递大块文本”改成“围绕稳定研究资产协作”。这种模式更接近人类研究同事的合作方式：资料有位置，草稿有版本，证据能追踪，疑点能保留。第 8 节会继续展开上下文窗口压力和未来研究产物；本章先留下基础判断：Deep Research 要走向更长、更复杂的任务，就需要把中间材料外部化到可读、可写、可审查的工作空间里。

4 Skills Library：组织知识如何进入智能体

Deep Research 解决的是一条研究管线如何协作。下一步问题更组织化：如果 Google 或 DeepMind 内部有大量团队、工具、代码库、流程和专家经验，智能体怎样获得这些能力？KP 的回答是，内部正在投入建设一个巨大的 skills library，让员工能把经验封装成可复用技能，帮助大家更快、更好地完成工作。

4.1 技能是专家经验的可复用封装

本节里的 skills library，译为“技能库”，指一组可被智能体调用的任务能力集合。一个 skill 可以是提示词模板、操作步骤、CLI 工作流、工具使用说明、代码库约定、评测方法或专家经验的组合。它的目标是把“某个专家知道怎样做”变成“智能体遇到类似任务时可以调用的能力”。

术语定义：技能库

技能库是组织知识的能力货架：专家把某类任务的判断标准、步骤、工具、输入输出格式和验收要求封装成 skill，智能体在合适任务中调用它。它把个人经验从口头传授和临场指导，变成可发现、可复用、可维护的操作单元。

技能的价值可以用一个简单类比说明。新人加入一个大型代码库时，真正卡住他的往往已经超出“会不会写 Python”这种通用能力，具体表现为“不知道本仓库如何跑测试”“不知道这个团队的 review 标准”“不知道某个内部工具应该用哪个入口”。专家的一句话可能抵得上新人半天搜索。skill 就是把这些“一句话背后的经验链条”写成智能体能执行的说明书。

KP 说技能库正在帮助人们以更快速度交付好代码。这里的重点不只是速度。好 skill 会把质量门槛也带进去：代码风格、内部 API 使用方式、测试要求、失败时的排查路径、输出报告格式，都可以成为 skill 的一部分。没有技能库时，智能体每次都像临时上岗；有技能库后，智能体至少能先读到组织积累下来的作业指导书。

4.2 Agent Sprawl: 规模组织里的能力失控风险

技能越有用，增长越快，风险也越快出现。KP 用了一个很直白的词：skills sprawling out of control。outline 将这个治理问题归到 agent sprawl，即智能体能力蔓延。这里的 sprawl 特指无控制增长：技能、代理、工具包装、任务变体越来越多，质量、归属、发现和维护开始失控。正常增长强调能力覆盖扩大；无控制增长会让能力目录本身变成新的故障源。

Agent sprawl 的风险

Agent sprawl 指智能体能力资产的无序扩张：相似 skill 重复出现，过期 skill 继续被调用，没人维护的工具包装仍在入口里，低质量流程和高质量流程混在一起。结果是智能体看似能力很多，实际选择成本变高、失败路径变多、责任归属变模糊。大组织里这类问题已经超出小洁癖范畴，属于生产系统风险。

为什么大组织特别容易出现这种风险？原因很硬。第一，贡献者多，每个团队都想把自己的流程写成 skill。第二，任务相似但细节不同，容易产生十几个只差一点点的变体。第三，内部工具会更新，旧 skill 会变成半可用状态。第四，智能体调用 skill 时不一定能可靠判断哪一个是最初、最权威、最适配的版本。人类遇到这种情况还能问同事；智能体如果拿错 skill，就会把错误流程执行得很认真。

可以把技能库看成一座城市的道路系统。道路太少，大家无法到达目的地；道路不断私搭乱建，没有路牌、没有维护、没有废弃机制，交通也会瘫痪。技能库治理的目标是让贡献进入可用地图：新路进入地图前经过命名、测试、维护和淘汰机制，贡献热情由此转成可运行的组织能力。

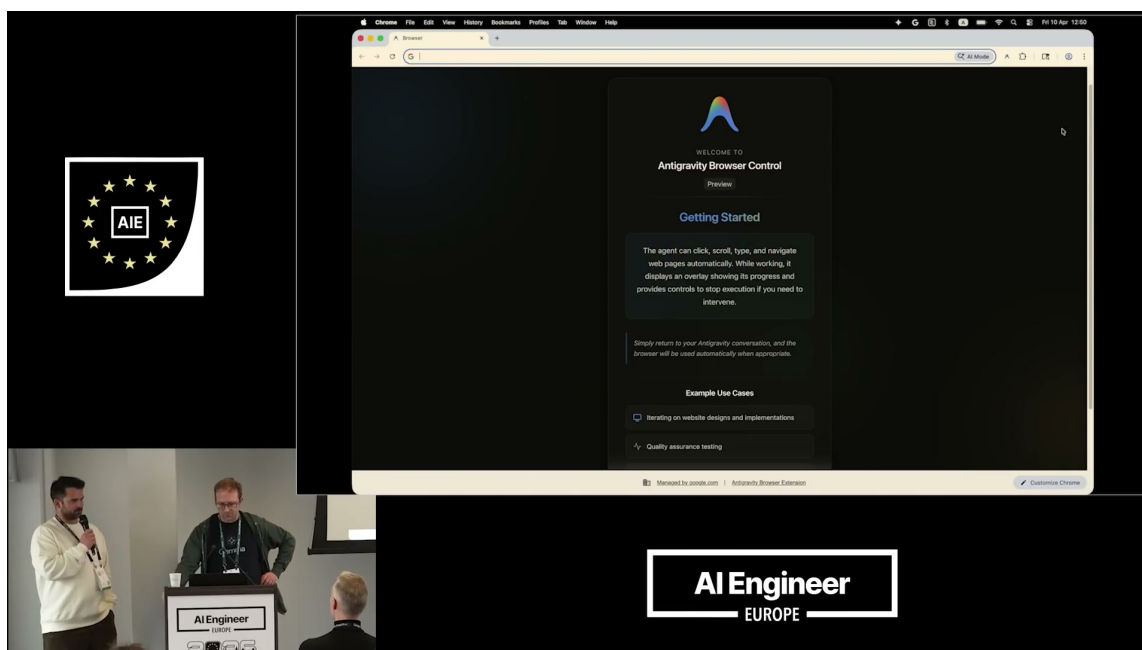


图 6: 技能库与 agent sprawl 讨论段落的现场上下文画面；视频没有展示可读的技能库列表或治理界面，因此该图主要提供来源锚点。⁵

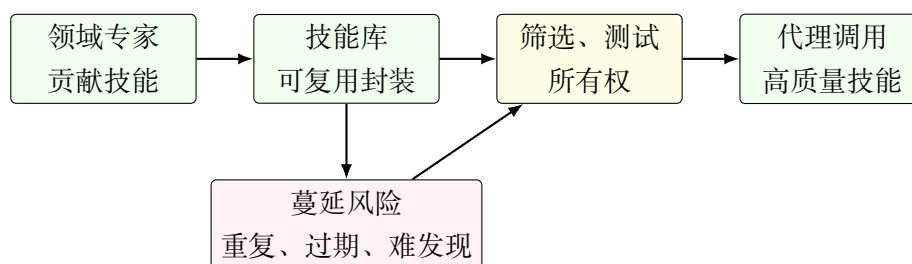


图 7: 根据 00:05:35–00:07:45 关于 skills library 与 agent sprawl 的讨论重绘的说明性示意图：专家经验进入技能库后，必须经过筛选、测试和所有权管理，才能避免能力蔓延拖垮可发现性与质量。

4.3 技能筛选与“达尔文式”治理

KP 说他们正在改进这些 skills，确保只有最好的 skill survive，并形容为 almost Darwinian nature。这个说法很关键，因为它暗示技能库治理不能只靠“大家自觉写好”。在大规模组织中，能力资产需要筛选压力：好 skill 被更多使用、维护和推广；差 skill 被标记、降权、合并或移出入口。

“达尔文式”治理可以拆成四个环节。第一是进入门槛：skill 至少要说明适用场景、输入输出、依赖工具、失败处理和验收方式。第二是使用反馈：系统要知道哪些 skill 被调用，成功率如何，常见失败点在哪里。第三是维护责任：每个高价值 skill 都要有人或团队负责更新。第四是淘汰机制：重复、过期、低成功率、无人维护的 skill 不能长期占据推荐入口。

这里也能和 Antigravity 演示接上。视频中游戏任务继续运行时，观众能看到蓝色边框表示 Antigravity 正在控制游戏；随后它查看 DOM，尝试理解交互区域，并在 scratchpad 中记录自己正在尝试的行为。这段画面虽然属于上一章 demo 的延续，却给技能库一个具体锚点：能力必须变成可观察的执行轨迹，治理才有抓手。一个 skill 是否有效，不能只看它的文档写得漂亮，还要看它在真实任务里调用了哪些工具、改了哪些文件、留下了怎样的中间记录、最终是否通过人类或自动检查。

⁵ 视频画面时间区间：00:06:00–00:06:32。

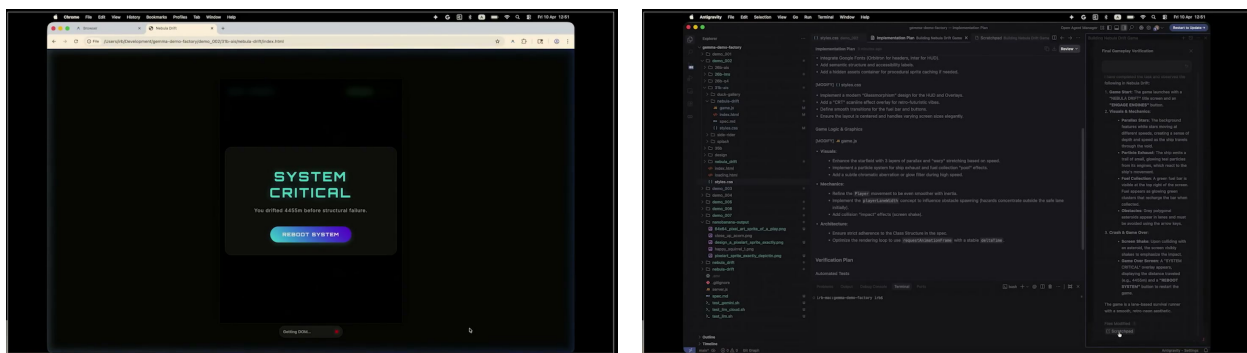


图 8: Antigravity 控制浏览器中的游戏，并在 IDE 侧留下 scratchpad 与验证轨迹，形成可回看的协作记录。⁶

技能治理要看“能力生命周期”

一个 skill 的生命不止创建。它会经历提交、审核、发布、调用、失败修复、版本更新、合并和下线。缺少生命周期视角，技能库会变成杂物间；拥有生命周期视角，它才像工程组织里的标准作业系统。

大组织的难点还在于“专家经验”常常带有上下文。某个团队的构建命令、代码风格、发布流程，对另一个团队可能完全不适用。高质量 skill 应该把适用边界写清楚，例如“只适用于某仓库”“只适用于某语言栈”“需要某内部权限”“需要先运行某环境检查”。边界写得越清楚，智能体越不容易把局部经验误当成全局规则。

4.4 本章小结

本章定义了 skills library 与 agent sprawl。技能库是把专家经验封装成可复用能力的组织机制：它让智能体在任务开始前就能拿到流程、工具、标准和验收要求。对 Google 这种规模的组织来说，这类能力货架可以显著降低重复摸索成本，帮助员工更快交付质量更稳定的代码和工作成果。

但技能库一旦扩张，就会带来 agent sprawl：重复 skill、过期 skill、无人维护 skill、低质量 skill 和命名混乱会让智能体能力表面繁荣、实际失控。KP 所说的“达尔文式”筛选，核心是用进入门槛、使用反馈、维护责任和淘汰机制管理技能生命周期。真正可规模化的技能库，必须同时服务能力复用与能力治理；只追求“更多 skill”，迟早会把智能体平台拖进选择成本和质量债务里。

5 Google 规模下的硬约束：Token、Quota 与计算资源

前几节把问题从单个演示推进到组织级能力复用：Antigravity 让一个人可以同时启动多个任务，技能库让专家经验可以被更多智能体调用。到了 Google 和 DeepMind 的规模，真正压到系统上的问题就变得很朴素：每一次规划、工具调用、浏览器观察、DOM 检查、报告生成和人工复核提示，都会转化为 token、模型调用和计算资源。讲者把这个压力概括为一个短语：agentic workflows are token hungry，也就是智能体 workflow 天然“吃 token”。

⁶视频画面时间区间：00:06:40–00:07:28。

5.1 为什么智能体工作流特别吃 Token

普通聊天的一次请求通常接近一个问答回合：用户输入问题，模型输出答案。智能体工作流的形态更像一个小型施工队：先读任务说明，再制定计划，然后调用工具，读取工具结果，修正计划，继续执行，最后产出报告或代码变更。每一步都要把当前目标、历史上下文、工具输出、状态摘要和下一步意图重新塞进模型输入。这样一来，token 消耗会沿着步骤数增长，且每个步骤还会携带额外的运行时开销。

这里的 token quota 指一个用户、团队或工作流在某段时间内可消耗的模型 token 预算。它在生产系统里同时承担成本控制、容量保护和公平调度三种职责，远比单纯计数更硬。一个简单直觉是：智能体越会做事，越容易把一次“帮我完成这个任务”拆成一串模型调用；能力提升会带来调用链拉长，调用链拉长又会扩大资源账单。

Token 配额的可计算表达

Token 配额消耗可以用下面的教学公式压缩表达：

$$Q = \sum_i (p_i + c_i + o_i)$$

- Q ：一次智能体工作流最终消耗的 token quota。
- i ：工作流中的步骤编号，例如计划、工具调用、观察、修正、总结。
- p_i ：第 i 步输入给模型的 prompt/input tokens，包含指令、上下文、工具结果摘要等。
- c_i ：第 i 步模型生成的 completion/output tokens，包含计划、代码、解释、下一步动作等。
- o_i ：第 i 步的额外开销 tokens，例如轨迹摘要、scratchpad、状态压缩、工具 schema、审计记录和报告骨架。

这个式子强调工程结构，精确计费口径留给具体平台实现：智能体的成本来自“多步骤乘以每步上下文”，并且工具越多、轨迹越可审计， o_i 往往越不能忽略。

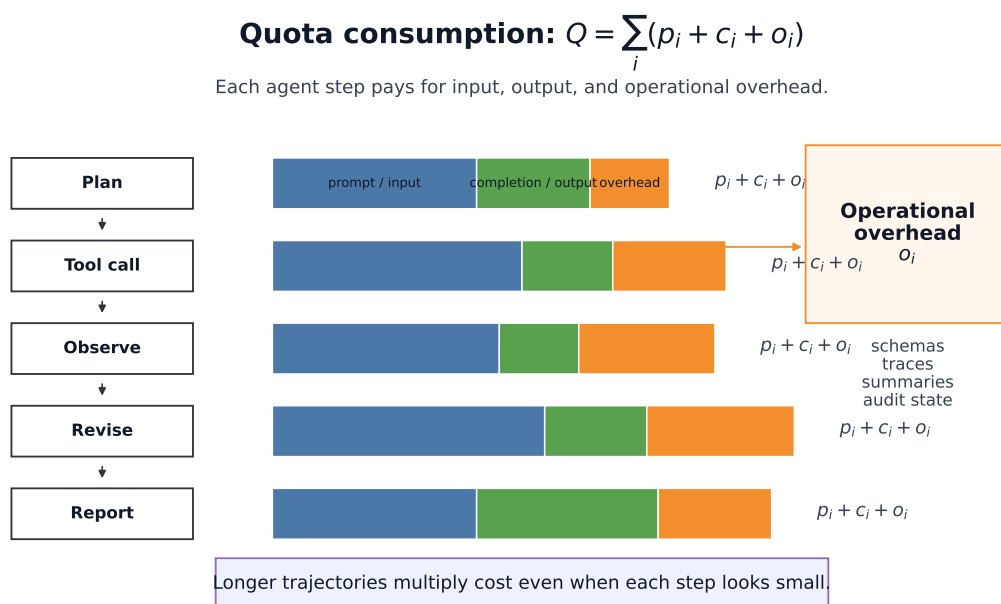


图 9: Token 配额消耗的教学示意图。根据 00:07:52–00:09:44 关于 agentic workflows are token hungry 的讨论重绘，强调一次智能体运行会在每个步骤同时消耗 prompt/input tokens、completion/output tokens 与轨迹、schema、摘要等 overhead tokens。

一个容易被低估的细节是 o_i 。在传统 API 调用里，开发者容易只盯着输入和输出；在智能体平台里，系统还要记录计划、工具结果、浏览器状态、文件 diff、失败原因、用户审批点和最终报告。这些材料对可观测性和审计很有价值，但它们不会免费存在。它们像物流系统里的包装、标签和追踪单号：没有它们，规模化配送会失控；有了它们，每一件货物的运输成本都会上升。

5.2 按用户与团队管理配额

讲者提到，按用户或按团队管理 quota 是眼前非常重要的工作。这个说法背后有一个现实：Google 这样的组织里，资源消耗很少只属于“某个功能”。同一个 Deep Research 能力可能在 Gemini app 面向外部用户，也可能被内部员工用于研究、编码、调试和文档生成。若只从全局总量看资源池，某个团队的实验、某个内部 harness 的评测，或者某位强用户的多任务并发，都可能在短时间内挤压其他任务。

按用户配额强调公平性：一个人的密集使用不能让其他人的 workflow 整体降速。按团队配额强调责任边界：团队知道自己拥有多少预算，也要决定把预算花在交互式开发、批量研究、评测回归还是生产功能上。两者结合后，平台才能回答三个实际问题：谁在消耗资源，消耗发生在哪类 workflow，超出预算时应该限制个人、团队、模型层级还是任务类别。

这里要区分 token quota 与 Section 12 将要展开的 quota management。Section 5 只定义资源经济学：为什么要有配额、配额保护什么、成本如何形成。至于配额耗尽时怎样提示用户、怎样从 Pro 模型切换到 Flash 或本地模型、怎样避免任务静默停住，那属于后续的连续性与 UX fallback 问题。

5.3 在本地模型、便宜模型与强模型之间分工

讲者特别提到模型混用。字幕中出现的具体型号“Gemma 4”疑似 ASR 或现场口语不稳，本文只保留机制层面的说法：本地或自托管模型从中心化 token quota 的角度看接近“免费”，但仍然消耗用户或团队拥有的 GPU/TPU；更强的模型则适合放在智能体系统中特定、高价值的环节。这个分工非

常关键，因为智能体 workflow 并不要求每一步都使用最强模型。

一种务实的分层方式如下。本地或自托管模型适合做高频、低风险、可重复的工作，例如日志初筛、格式转换、候选文件过滤、简单摘要、局部代码解释。Flash 一类便宜且响应快的模型适合承担交互密集的步骤，例如生成初步计划、整理工具输出、给浏览器观察结果做短摘要、维护轻量状态。强模型适合处理需要深推理和高准确度的节点，例如跨文件架构判断、复杂 bug 定位、关键代码生成、失败复盘和最终报告。

Model mix and low-cost harness evaluation

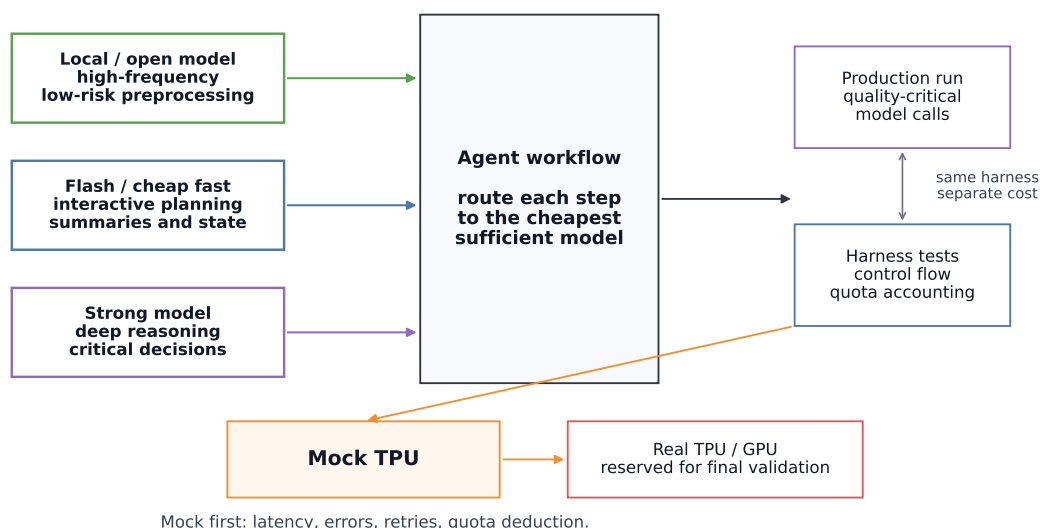


图 10: 模型分工与 mock TPU 低成本评测示意图。根据 00:07:52–00:09:44 关于本地/开源模型、Flash/低成本快速模型、强模型和 mock TPU 的讨论重绘；图中用“本地/开源模型”避免把可能来自 ASR 的具体型号写死。

模型混用的资源逻辑

模型混用的核心机制是“把强模型放在瓶颈判断点，把便宜模型放在高频搬运点，把本地模型放在可承受误差的预处理点”。这会把资源压力从“每一步都贵”改造成“只有关键步骤贵”。在数学上，可以把总成本理解成多类调用的加权和：高频步骤的单价哪怕很低，累计后也可能超过少数强模型调用；因此平台要同时看单步质量、调用频率和失败重试率。

这种分工还会改变产品策略。若一个智能体在 30 个步骤里只有 3 个步骤真正需要强模型，平台就可以把其他 27 个步骤放到更便宜的推理路径上。这样做的收益并不只是省钱；它还减少强模型容量被低价值步骤占用的概率，让真正困难的任务在高峰期仍然有机会拿到好模型。

5.4 Mock TPU 与低成本评测环境

讲者随后把话题转到 evaluation，并给出一个很具体的工程方向：使用 mock TPU 来测试 harness 和 agentic flow，避免为了验证流程本身烧掉大量 TPU hours。这里的重点不在 TPU 这个硬件名词本身，而在“评测对象”要拆开。智能体系统至少有两层需要评测：一层是模型能力，另一层是 harness 是否能正确调度步骤、传递状态、处理工具结果、记录轨迹和结束任务。

如果每次评测 harness 都真实占用昂贵 TPU，那么评测成本会反过来限制迭代速度。mock TPU

的作用类似飞行模拟器：它不提供真实飞行的全部物理风险，却能反复测试仪表、流程、指令、异常处理和操作员反应。对于智能体平台，mock TPU 可以模拟预测请求、容量限制、延迟、错误码和特定返回模式，使工程师在不消耗大量真实算力的情况下验证控制流。

这并不意味着真实评测可以省掉。mock 环境更适合回答“流程是否连得起来”“失败时是否记得清楚”“任务是否会循环”“配额是否被正确扣减”这类问题；真实模型和真实硬件仍然要回答“答案质量是否足够”“长链路推理是否稳定”“复杂任务是否真的完成”这些问题。两者的边界划清后，评测体系才不会把所有成本压到最贵的资源上。

5.5 本章小结

Section 5 的核心结论是：在 Google 规模下，智能体的瓶颈不能只用模型能力解释。智能体工作流会因为多步骤、工具调用、轨迹记录和上下文回传而迅速放大 token 消耗；token quota 因而成为成本、容量与公平性的共同控制面。按用户和团队管理配额，是为了让资源账单能对应到真实组织边界。本地或自托管模型、Flash 类低成本模型和强模型的分工，则是在不同质量、成本和延迟之间做工程分配。mock TPU 的价值在于把 harness 流程评测从真实 TPU 小时中剥离出来，让平台可以更频繁地测试控制流，同时把昂贵算力留给真正需要质量判断的环节。

6 Per-user Behavior：一个强用户怎样拖垮系统

Section 5 讨论的是全局资源账本。Section 6 把镜头缩到单个用户：当一个强用户同时启动多个实例、多个代理和长时间任务时，系统会遭遇怎样的突发负载。这里的 per-user behavior 指单个用户的负载画像，包含启动了多少任务、每个任务运行多长、调用哪些模型、是否反复重试、是否并发运行，以及这些行为在时间上是否集中爆发。

6.1 多实例、多代理与突发负载

现场问题很直接：随着智能体越来越能干，用户会让它们并行工作。一个强用户可能把智能体从单个聊天窗口扩展成一支“100 人团队”：多个实例同时修代码、跑评测、查资料、生成报告、开浏览器、读 DOM、写文件。每个代理看起来都只是一个任务，合在一起就可能形成集群层面的流量尖峰。

突发负载的危险在于它不一定来自恶意攻击。一个用户只是更熟练、更依赖自动化、更愿意把任务拆出去，就足以让系统看到类似压测的形状。对平台而言，负载画像要看并发度、持续时间和单任务成本三项叠加。并发度决定瞬时峰值，持续时间决定资源占用窗口，单任务成本决定每条执行链的燃烧速度。

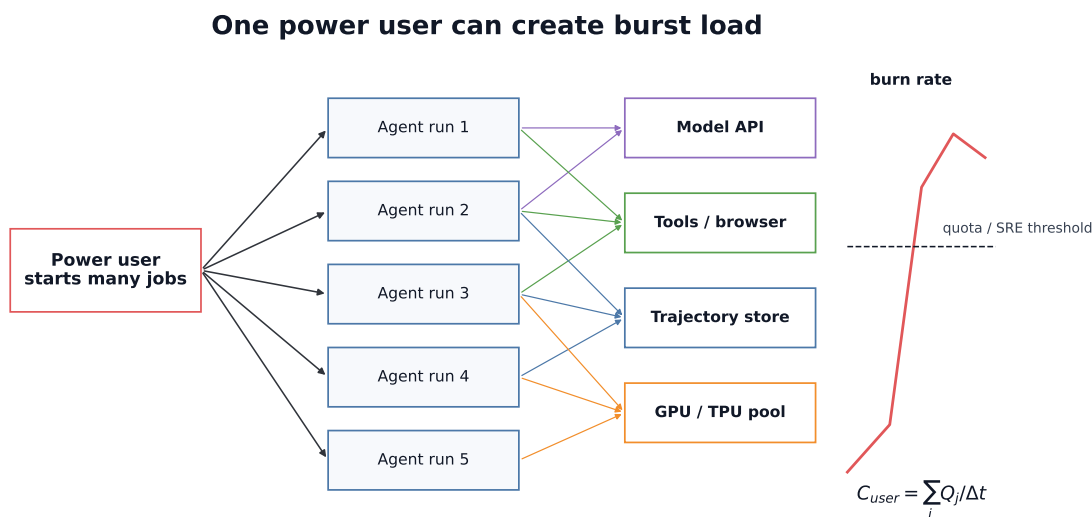


图 11: 强用户并发启动多个 agent run 后形成 burst load 的系统示意图。根据 00:09:44–00:13:12 关于 per-user behavior、粗粒度 quota 和 SRE spike 监控的讨论重绘，突出单用户 burn rate 会叠加到共享模型、工具、轨迹存储和计算资源上。

强用户不等同于恶意用户

一个常见误解是：强用户拖垮系统等同于滥用或攻击。更准确的工程判断是，强用户把产品承诺用到了极限。他们可能完全按照界面允许的方式启动任务，却因为并发实例、多代理协作和长任务链路，把平台推到容量边界。治理这类行为时，系统既要防止单点用户压垮共享资源，也要保留高价值用户真正需要的批量工作能力。

这个区别很重要。若把所有高消耗都当成坏行为，产品会惩罚最能发现价值的用户；若完全放任高消耗，系统会让其他用户替强用户承担延迟和失败。平台真正要做的是把 per-user behavior 从模糊印象变成可测量的负载画像。

6.2 粗粒度限额的现实做法

讲者给出的现状很坦率：目前有些场景仍然靠比较 brute force 的 quota 来处理。也就是说，当强用户消耗到某个点，系统或运维人员只能要求任务停止，或者直接限制继续执行。这种做法听起来粗糙，但在资源紧张、需求增长快、模型调用链复杂的阶段，它常常是第一道可落地的保护阀。

要把这个保护阀讲清楚，可以引入 per-user quota burn rate，即单个用户在一个时间窗口内燃烧配额的速度：

单用户消耗速率

单用户负载可以用下面的教学公式表达：

$$C_{user} = \sum_j Q_j / \Delta t$$

- C_{user} ：某个用户在观测窗口内的 quota 消耗速率。

- j : 该用户启动的 job 或 agent run 编号。
- Q_j : 第 j 个 job 消耗的 token quota, 可由 Section 5 的 Q 公式估算。
- Δt : 统计窗口长度, 例如 1 分钟、15 分钟或 1 小时。

当 C_{user} 在短窗口内急剧上升时, 平台看到的是“一个用户正在把多个贵请求叠到同一个时间段”, 问题已经超出单个请求变贵的范围。这正是突发负载最容易伤害共享系统的地方。

粗粒度限额的优点是简单、可解释、能立刻保护容量。它的代价也很明显: 用户可能只看到任务被截停, 却不知道是哪个实例、哪条代理链或哪类模型调用触发了限制。这个问题会在后续 Section 7 的 observability 中继续展开: 若看不到轨迹, 平台只能粗暴截停; 若能看到轨迹, 平台就有机会定位循环、异常重试和无效工具调用。

6.3 订阅制遇到智能体工作流的张力

讲者还指出一个商业层面的张力: agentic systems so token hungry, 传统订阅模型很难自然覆盖。订阅制的直觉是用户按月付费, 平台提供一个大致可预期的服务包。智能体系统打破了这个平衡, 因为用户一次点击可能触发几十次模型调用、多个工具执行和长时间评测。用户看到的是“我让它做一个任务”, 平台承担的是“一串持续燃烧 token 和计算的工作流”。

这会带来一个简单但刺耳的经济问题: 若订阅价格固定, 而强用户的边际消耗远高于普通用户, 那么平台需要在三种方向中取舍。第一, 限制强用户并发和任务长度, 以保护平均成本。第二, 引入更粒度的用量计费, 让高消耗更接近真实成本。第三, 通过模型混用、缓存、mock 环境和更好的 harness 设计降低单位任务成本。现实系统通常会同时使用这三类手段, 只是比例不同。

现场提到 Anthropic 对某些高消耗 agentic 使用做限制, 值得在这里作为行业信号理解: 当 agent 可以自动化地产生大量 token 需求时, 服务提供方会重新审视“无限用”或“高上限订阅”的经济可持续性。这里不需要把问题归咎于某个用户群体; 关键在于 agent 把人类时间成本转成机器时间成本, 把等待和操作外包给模型后, 平台资源账本会以更快速度显形。

6.4 SRE 视角: 什么时候算太多

Ian 用一个内部经验说明资源边界如何被发现: 当某个 job 在某个 cluster 上跑得过重时, 会有 24/7 监控 spikes 和 graphs 的 SRE 团队联系相关人员, 请他们停止那项工作。这个例子把抽象的 quota 变成了生产现实。平台可以给用户提供友好的界面, 但底层仍然要面对集群、容量、峰值、排队和抢占。

从 SRE 视角看, per-user behavior 的目标是提前识别哪些行为会制造不可接受的峰值, 而非给用户贴标签。一个成熟系统通常会逐步从“人来发现峰值并叫停”走向“系统自动识别 burn rate、并发数、长尾任务和异常循环”。不过在新型智能体工作流快速扩张的阶段, 人类 SRE 的 24/7 监控仍然是最后防线。

6.5 本章小结

Section 6 的核心结论是: 单个强用户足以制造系统级问题, 因为智能体把个人操作放大成多实例、多代理和长任务的并发负载。per-user behavior 要被定义成可测量的负载画像, 避免滑向对用户动机的猜测。 C_{user} 公式帮助平台看见某个用户在单位时间内燃烧了多少 quota。粗粒度限额在现实中有必要,

但它会牺牲解释性和细粒度控制。订阅制与 token hungry agent 系统之间存在经济张力，因为固定价格难以覆盖高度不均匀的资源消耗。SRE 监控提醒读者：智能体平台最终仍然运行在有限集群和有限算力之上，任何“自动化能力”都要接受容量边界的约束。

7 Observability：看见智能体轨迹，才能调试智能体

上一节讨论了单个强用户如何通过多实例、多任务和长时间运行把系统推到资源边界。仅有配额限制还不够；当一个智能体系统已经在生产环境里跑起来，工程团队还必须回答一个更难的问题：这一次运行到底发生了什么？在 00:13:12–00:14:46 的回答中，讲者把这个问题落到两个内部设施上：统一后端的层级化排查界面，以及面向编码任务的轨迹存储。

本节负责定义两个术语。可观测性（observability）指系统允许工程师从工作流层级一路下钻到原始模型请求、工具调用和中间状态，从而解释一次智能体运行的能力。轨迹存储（trajectory store）指保存智能体步骤、工具调用、观察结果、模型请求、临时状态和失败点的记录系统。二者的关系像医院里的影像系统和病历系统：前者让医生定位正在看的层级，后者保留病程中的关键节点。对于智能体平台，调试对象经常超出单个函数返回值，呈现为一串会计划、会试错、会调用工具的行动链。

可观测性的下钻含义

可观测性不能被泛化成“多打日志”。在这段讨论里，它的关键含义是层级化下钻：用户 query 先出现在统一 UI 中，工程师再按系统组件、子步骤、工具交互和模型请求逐层展开，必要时看到 raw predict request。没有这条下钻路径，团队只能看到“任务失败了”；有了这条路径，团队才可能判断失败发生在意图理解、计划拆分、工具返回、浏览器状态、模型调用，还是后处理环节。

7.1 统一后端与层级化排查界面

讲者提到，Google 内部有一个用于大量智能体任务的统一 agent backend system。每当用户向托管在这个系统上的智能体发出 query，这次请求会自动出现在一个内部 Web UI 中。这个设计的价值不在于界面本身华丽，而在于它把一次智能体运行变成可展开的树。

可以把这棵树理解为四层。第一层是用户 query，也就是“这次任务的入口”。第二层是工作流组件，例如规划器、工具执行器、检索模块、浏览器控制模块、代码编辑模块。第三层是每个组件内的具体事件，例如一次文件读取、一次 DOM 检查、一次测试运行或一次网页观察。第四层是 raw predict request，也就是实际送到模型端的请求内容、上下文片段、工具 schema、系统提示约束和模型返回。越往下，信息越接近运行时真相；越往上，信息越接近产品语义。

这种层级视图还有一个隐含收益：它让跨团队排查有共同坐标。前端团队关心用户看见了什么，平台团队关心调度和工具权限，模型团队关心 prompt 与输出，SRE 团队关心延迟、错误率和资源尖峰。如果所有人都围绕同一个 query id 和同一条展开路径讨论，争论会从“像是模型问题”变成“第 17 步工具返回空结果后，第 18 步 raw predict request 仍把旧观察当成事实使用”。后者才是可以修的工程问题。

Observability drill-down plus trajectory store

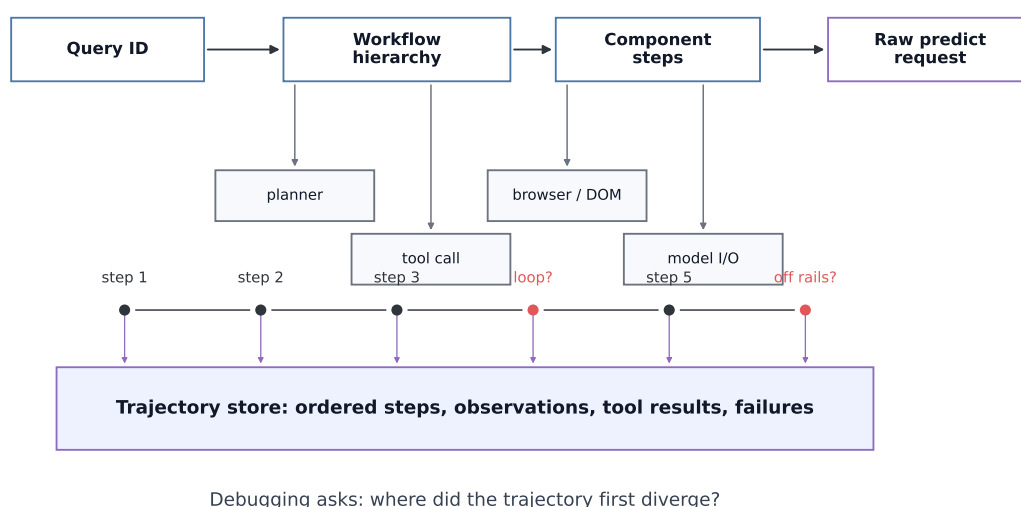


图 12: 可观测性下钻路径与 trajectory store 的诊断示意图。根据 00:13:12–00:14:46 关于 query、层级化排查、raw predict request 和轨迹存储的讨论重绘，展示工程师如何从产品语义一路下钻到模型请求与有序步骤记录。

7.2 Trajectory Store: 保存步骤、循环与失控点

轨迹存储更偏向长链任务，尤其是编码智能体。讲者强调，编码场景里可能出现 “huge number of steps”。一个编码智能体会读文件、写补丁、运行测试、查看错误、修复错误、再次运行测试，还可能打开浏览器或检查 DOM。每一步都可能改变后续步骤的输入条件。若只保存最后的失败消息，调试者会像只拿到一部电影最后一帧：知道结局糟糕，却不知道剧情何时转向。

轨迹存储要解决的核心问题是定位 “失控起点”。循环（loop）通常表现为智能体反复执行近似相同的动作，例如反复修改同一段代码、重复运行同一个失败测试、不断重开浏览器却没有吸收新观察。off rails 则表示智能体偏离任务轨道，例如开始解决未被要求的问题、把工具错误解释成业务事实、或把临时观察当成最终结论。二者都需要时间顺序证据：第几步开始重复？第几步之后假设发生漂移？哪一次工具返回导致模型转入错误分支？

轨迹存储的飞行记录仪类比

轨迹存储的教学定义：它是智能体运行的 “飞行记录仪”，记录步骤、工具调用、模型请求、观察结果和失败点。它的价值在于还原因果链，而非事后堆叠日志。诊断循环时，工程师关心的是重复模式的起点；诊断 off rails 时，工程师关心的是任务意图、外部观察和模型内部假设第一次分叉的位置。

7.3 从 Predict Request 到 workflow 诊断

raw predict request 是这条排查链的最底层证据之一。它通常包含模型实际看到的指令、历史摘要、工具定义、当前观察以及预期输出格式。很多智能体问题在产品层看起来相似，例如 “没有完成任务” “答非所问” “测试失败后卡住”。一旦下钻到 raw predict request，问题会被分解成更具体的几类。

第一类是输入污染：模型看到的上下文里混入过时文件、旧错误、被截断的需求或错误摘要。第二类是工具语义错配：上层以为工具返回的是事实，模型实际收到的是不完整文本或异常包装。第三类是层级摘要损坏：为了省 token，上层把长轨迹压缩成摘要，但摘要抹掉了关键约束。第四类是控制策略问题：系统提示允许模型继续尝试，缺少“连续失败后必须停止并请求人类确认”的边界。

从这个角度看，observability 与 evaluation 有清楚分工。前者解释“这次为什么这样走”，后者判断“这类任务总体能不能信任”。一个平台缺少可观测性，即使评测分数不错，线上事故也会难以复盘；一个平台只有可观测性，没有评测，则会陷入逐案救火，无法判断改动是否整体提升可靠性。

7.4 本章小结

本节把可观测性定义为多层级检查智能体运行的能力，把轨迹存储定义为保留智能体行动链的记录系统。统一后端 UI 让工程师从用户 query 下钻到组件、步骤和 raw predict request; trajectory store 让工程师沿时间顺序定位 loop 与 off rails 的起点。讲者透露这些系统目前是内部定制设施，这一点也说明了现实状况：规模化智能体平台离不开专门的排查基础设施，普通聊天记录无法承担这类诊断负载。

8 Deep Research 的下一步：让管线成员像研究同事一样协作

本节延续第 3 节已经定义过的 Deep Research 智能体，不重新定义产品边界。这里关注讲者在 00:14:46–00:16:10 提出的下一步：用同一套 harness 编排更深的研究任务，并把管线中的各个元素组织成共享 workspace 中的协作者。这个想法的出发点很朴素：深度研究任务会产生大量上下文，若每个阶段都把大段文本 blob 传给下一阶段，成本、延迟和上下文压力会同步上升。

8.1 上下文窗口如何被长研究任务消耗

上下文窗口（context window）是模型在一次交互或一次工作流片段中可关注的有限 token 容量。长研究任务会快速吃掉它，因为研究过程本身会不断扩张材料：搜索结果、网页摘录、候选论点、反例、引用片段、用户历史要求、工具返回、临时计划、最终报告草稿都会争夺同一个窗口。

可以用一个简单的预算式表达剩余上下文空间。这个式子属于对本段工程压力的教学抽象，讲者原文没有给出：

$$B_{\text{ctx}} = W - (I + H + T + E)$$

- B_{ctx} : remaining context budget, 即当前还可用于继续思考、检索和整合的剩余上下文预算。
- W : model context window, 即模型一次可接收和关注的总上下文窗口。
- I : instructions, 即系统指令、开发者指令、任务要求、格式约束和安全边界。
- H : conversation/history, 即用户与系统已经形成的对话历史、计划历史和必要的运行摘要。
- T : tool outputs or research text, 即搜索结果、网页摘录、文档片段、工具返回和中间研究材料。
- E : expected answer budget, 即为最终答案、引用、结构化输出或报告草稿预留的生成空间。

这个式子的直觉很简单：窗口 W 是固定容量的书桌， I 、 H 、 T 、 E 都在往桌上放材料。桌面剩余空间 B_{ctx} 变小后，模型会更依赖摘要、截断和选择性注意。长研究任务的危险在于 T 会膨胀得特别快；搜索越多，材料越多，后续阶段越需要搬运和压缩。若每个管线成员都拿到一整个大 blob，系统会在“传递材料”上花掉大量上下文，而真正用于判断、比较和写作的空间被挤压。

上下文耗尽的间接症状

上下文耗尽经常以间接症状出现：模型开始忽略早期约束、重复已经做过的检索、引用不再对应材料、报告结构突然变粗糙，或把中间假设写成确定结论。它看起来像“模型变笨”，实际常由上下文预算被材料搬运和历史负担耗尽触发。对长研究任务来说，token 费用和认知带宽会同时被消耗。

8.2 工作空间协作模型

讲者提出的方向是让系统中的每个元素“more like a collaborator as part of a workspace”。这里的 workspace 可以理解为共享文件系统、共享状态层或共享研究桌面。关键变化是：管线成员不必把所有材料塞进下一次模型上下文，可以把可复用产物放到工作空间里，后续成员按需读取、更新、引用和整理。

传统管线像传送带：检索阶段把一大包搜索文本交给筛选阶段，筛选阶段再把一大包摘要交给写作阶段，写作阶段继续把草稿交给审校阶段。共享 workspace 像研究小组的项目文件夹：有人维护来源清单，有人整理证据表，有人画信息图，有人写章节草稿，有人检查引用。成员之间通过稳定文件、结构化索引和可追踪产物协作，减少把全部背景重新塞进每次对话。

共享 workspace 的研究协作类比

共享 workspace 的类比：多人写论文时，很少有人把所有 PDF、笔记、图表和草稿反复复制到聊天窗口里。更自然的做法是维护一个共享文件夹、引用库、白板和任务清单。智能体协作也类似：检索者留来源和摘录，分析者留下证据矩阵，写作者读取必要材料，审校者标注风险。workspace 把“传大 blob”改成“按需读写共同产物”。

这种设计还能改变失败恢复方式。若一个阶段失败，系统可以回到共享 workspace 查看已经完成的材料，而不必重跑整条管线。若审校发现某个结论证据不足，可以在 workspace 中追踪到来源、摘录和中间判断，再要求检索成员补证据。若用户要求新增角度，系统也可以复用已有材料，只补充新增维度。对 Deep Research 这类任务，workspace 的价值不在于把答案写得更花哨，真正重点是把研究过程变成可累积、可检查、可重用的工程对象。

Deep Research: from blob pipeline to shared workspace

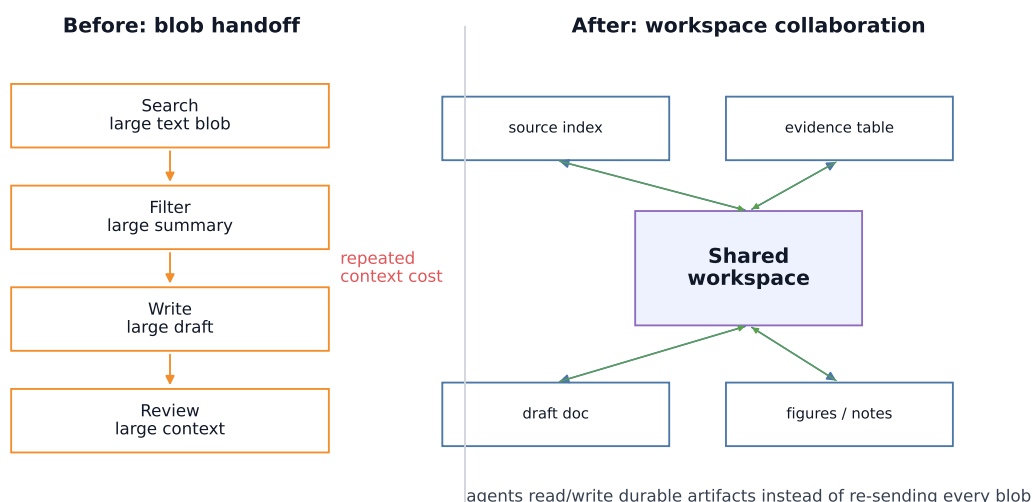


图 13: Deep Research 从大文本 blob 管线转向 shared workspace collaboration 的对比示意图。根据 00:14:46–00:16:10 关于 context window、research blob 和 workspace collaborator 的讨论重绘，强调共享产物能减少重复上下文搬运。

8.3 信息图、支撑材料与文档产物

讲者特别提到，这个方向会打开 infographics、additional support artifacts 和 documents 的潜力。这里的重点是产物形态从单一长文扩展为一组互相支撑的研究资产。长文报告负责叙事和结论，信息图负责展示结构，证据表负责支撑判断，附录文档负责承载细节，来源清单负责可追溯性。

这也解释了为什么同一套 Antigravity harness 能从编码任务推广到研究任务。编码任务有文件、测试、运行结果和审查记录；研究任务也可以有来源文件、证据矩阵、草稿、图表和审校意见。共同点是二者都需要一个能够承载中间状态的环境。如果智能体只能在一次上下文里“想完所有事”，它会受窗口限制；如果智能体能在 workspace 里留下结构化中间产物，它就更接近人类研究团队的协作方式。

8.4 本章小结

本节沿用第 3 节对 Deep Research 的定义，并扩展它的未来管线形态。长研究任务会让 T 项，也就是工具输出和研究材料，快速膨胀，进而压缩 B_{ctx} 。共享 workspace 的核心价值是减少 blob 搬运，让不同管线成员围绕可追踪产物协作。这样做可能带来更低成本、更快速度和更好结果，前提是 workspace 中的材料命名、版本、引用和权限都能被平台严肃管理。

9 Multi-agent System: 多轨并行与人工监督

在 00:16:10–00:17:45，听众把问题推向多智能体系统。讲者的回答很谨慎：Antigravity 的呈现方式里确实可以让多个智能体或多条工作轨道同时推进，但具体有哪些 sub-agents 在某个任务中协作，并不总是清晰暴露。这个回答很有价值，因为它没有把多智能体包装成全自动集群神话，并直接指出了多轨并行的现实边界：可并行、会重叠、部分不透明，需要人类监督。

本节定义 multi-agent system: 多智能体系统指多个智能体或多条智能体工作轨道在同一项目中并行工作、局部重叠、需要通信或需要共同接受监督的运行形态。这里的“多个”既可以是显式启动的多个 agent, 也可以是一个平台内部为任务拆出的若干工作轨。真正重要的问题不在数量本身, 重点在它们如何分工、如何共享状态、如何避免互相踩踏, 以及人类如何塑造方向。

9.1 Antigravity 中的多代理呈现方式

讲者描述的形态更接近“多个 simultaneous ones working on different tracks”。也就是说, 用户可以让不同轨道在同一项目中处理不同事项, 例如一个轨道修 UI, 一个轨道补测试, 一个轨道调查错误, 一个轨道更新文档。这种并行能节省等待时间, 也能让项目像流水线一样推进。

但并行有代价。多个轨道读写同一项目时, 状态会变化: 一个轨道修改了接口, 另一个轨道正在基于旧接口写测试; 一个轨道清理了文件结构, 另一个轨道还在引用旧路径; 一个轨道生成了新结论, 另一个轨道沿着旧假设继续执行。这里的 overlap 有两面性: 适度重叠能让任务互相校验, 过度重叠会制造冲突、重复劳动和调试噪声。

多智能体系统的不透明并行

多智能体系统的风险在于“不透明的并行”。如果平台只告诉用户“有很多代理在工作”, 却不展示每条轨道的目标、输入、输出、锁定资源和当前状态, 人类 supervisor 会失去控制感。此时并行不一定提高吞吐, 反而可能放大上下文耗尽、文件冲突、重复尝试和责任归属混乱。

9.2 Agent-to-agent Communication 的效率问题

讲者把未来问题概括为: 如何让 agent-to-agent communication 更高效。通信的目标并非让智能体互相聊天越多越好。通信也会消耗 token、引入延迟、传播错误摘要, 并扩大上下文压力。高效通信的目标是让必要信息在正确粒度上传递: 谁改了什么, 为什么改, 依赖什么, 阻塞点是什么, 哪些假设已经失效, 哪些产物可被复用。

可以把多智能体通信分成三种粒度。第一种是广播式状态, 例如“测试套件目前失败在 X”。第二种是点对点交接, 例如“搜索轨道把来源表交给写作轨道”。第三种是共享工件, 例如设计文档、任务清单、证据矩阵、补丁队列。工程上最可靠的方式通常会偏向共享工件, 因为它们可追踪、可审查、可回滚, 也更容易进入前一节的 workspace 模型。

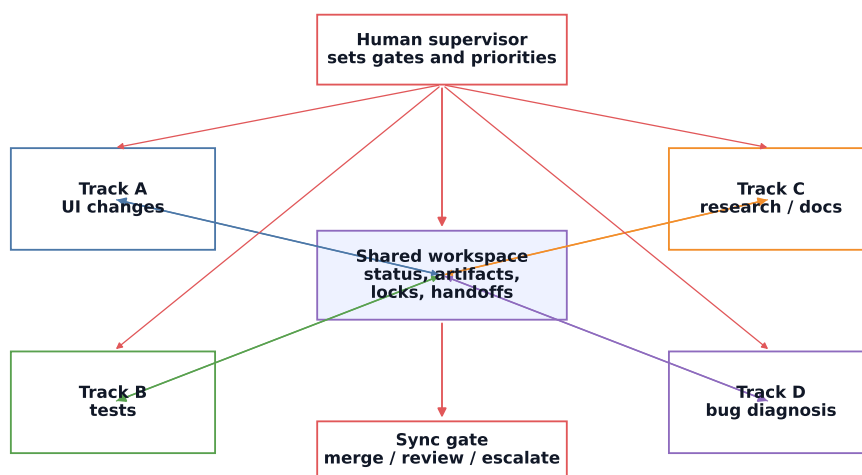
这里需要警惕一个常见误解: 多智能体数量增加不会自动带来线性效率提升。若每个 agent 都需要读取全部历史、询问其他 agent、等待协调、处理冲突, 系统可能出现通信开销吞掉并行收益的情况。用数学语言说, 若单轨工作量为 $O(n)$, 并行后理想情况接近 $O(n/k)$; 但通信和冲突成本可能增长为 $O(k^2)$ 级别的成对协调。真实收益取决于任务可拆分性、状态隔离和监督机制。

9.3 人类像装配线主管一样塑造协作

讲者用了一个很直观的比喻: 人类要像 digital assembly line 上的 supervisor。这个比喻比“人类批准按钮”更强, 因为 supervisor 不只在最后盖章, 还要塑造节奏、分工、质量门槛和异常处理。

在人类监督下, 多智能体系统至少需要四类控制。第一类是任务切分: 哪些轨道可以并行, 哪些必须串行。第二类是边界设置: 哪些文件、数据、工具或外部服务可以被某条轨道操作。第三类是同步点: 何时合并结果, 何时暂停等待评审, 何时把冲突升级给人。第四类是质量门槛: 测试、评测、引用检查、代码审查或用户验收通过后才能进入下一阶段。

Multi-agent topology with human supervision



parallel tracks help only when communication is structured and inspectable

图 14: 多轨并行、agent-to-agent communication 与人类 supervisor 的拓扑示意图。根据 00:16:10–00:17:45 关于 multi-agent system、parallel tracks 和 digital assembly line supervisor 的讨论重绘，强调并行收益依赖共享状态、同步门槛和人工监督。

这个 supervisor 角色也把第 7 节和第 8 节连接起来。没有 observability，人类看不到各条轨道正在做什么；没有 workspace，各条轨道只能用脆弱的上下文传话；没有监督策略，agent-to-agent communication 会变成新的噪声来源。多智能体系统的关键目标是让并行工作在可见、可控、可复盘的约束中产生增益，避免单纯放大每个 agent 的自由度。

9.4 本章小结

本节把 multi-agent system 定义为多个智能体或多条工作轨道并行、重叠、通信并接受监督的系统形态。讲者承认 Antigravity 中具体 sub-agent 细节有不透明之处，但指出了未来方向：更高效的 agent-to-agent communication，以及人类作为 digital assembly line supervisor 来塑造协作。多轨并行的工程收益取决于任务拆分、状态隔离、共享工件和监督节奏；缺少这些条件，并行会把上下文压力和调试难度一起放大。

10 Skills 与 MCP：能力接入的两条路线

前一节讨论的是多智能体系统怎样被人类监督。这个问题自然会推到下一层：智能体到底通过什么入口获得能力。现场提问把焦点放在 skills 和 MCP 上。KP 的回答很直接：他个人明显偏向 skills，再配合受护栏约束的 CLI 交互；MCP 在认证和外部工具接入上有价值，但在他的日常工作里，skills 加 guardrailed CLI 已经把大量任务变快了。

这里要守住一个边界。第 4 节已经定义了 skills library：它是组织专家经验的可复用封装。本节不重新定义它，只讨论当这种能力封装已经存在时，智能体还需要怎样连接外部工具、权限系统和命令行环境。

10.1 Skills: 把专家知识交给代理

KP 说自己 “team skills”，原因并不神秘。大组织里的专家知识往往藏在局部经验里：谁知道某类原始日志怎么排查，谁知道某个内部 CLI 怎么跑，谁知道某个代码库的测试和回滚规矩。把这些知识写成 skill 后，智能体和使用者的关系就像临时请到了一位熟悉该领域的同事。用户不必每次解释日志格式、命令参数和判断顺序，智能体也不必只靠通用模型能力猜测下一步。

这解释了 KP 为什么举 “debugging raw logs” 的例子。原始日志排查通常包含三类知识：第一，日志字段如何解读；第二，哪些异常模式值得优先看；第三，排查命令怎样组合。通用对话模型可以读文本，但如果没有 skill，它像拿到一堆散乱票据的人，只能从表面模式推断；有了 skill，它更像拿到一份内部排障手册，知道先过滤哪类事件，再查哪个服务，再把结果整理成什么格式。

comparison: Skills、MCP 与受护栏 CLI 的能力边界

Skills 负责把专家经验、步骤、判断标准和局部上下文交给智能体；MCP (Model Context Protocol) 负责把模型或智能体连接到外部工具与服务，尤其适合处理授权、工具发现和标准化访问；guardrailed CLI 负责让智能体在受控命令行中完成真实操作，例如读取日志、运行测试、调用内部脚本或生成诊断输出。三者关注点不同：skills 偏知识封装，MCP 偏协议化工具接入，受护栏 CLI 偏可审计执行。

KP 偏向 skills 的工程理由很硬。第一，skills 能吸收专家贡献。在 Google 这种规模的组织里，某个领域的最佳实践往往由真正做这件事的人写出来，质量高于平台团队凭空整理的通用模板。第二，skills 更接近任务意图。一个 “排查原始日志” 的 skill 可以同时包含背景知识、命令、失败分支和输出格式。第三，skills 容易同 CLI 组合。若任务最终要落到命令行，skill 可以告诉智能体该调用哪个命令、怎样加参数、如何解释返回结果。

这里的关键在于避免盲目信任命令行。guardrailed CLI 指的是暴露给智能体的命令行执行面带有明确边界、策略、提示和可审查操作。它像给实习工程师一间带门禁和摄像头的实验室：可以动手做事，但做事范围、危险动作、审批点和记录方式都要清楚。没有护栏的 CLI 会把智能体直接推向高风险执行面；有护栏的 CLI 才能把 “能运行命令” 变成产品级能力。

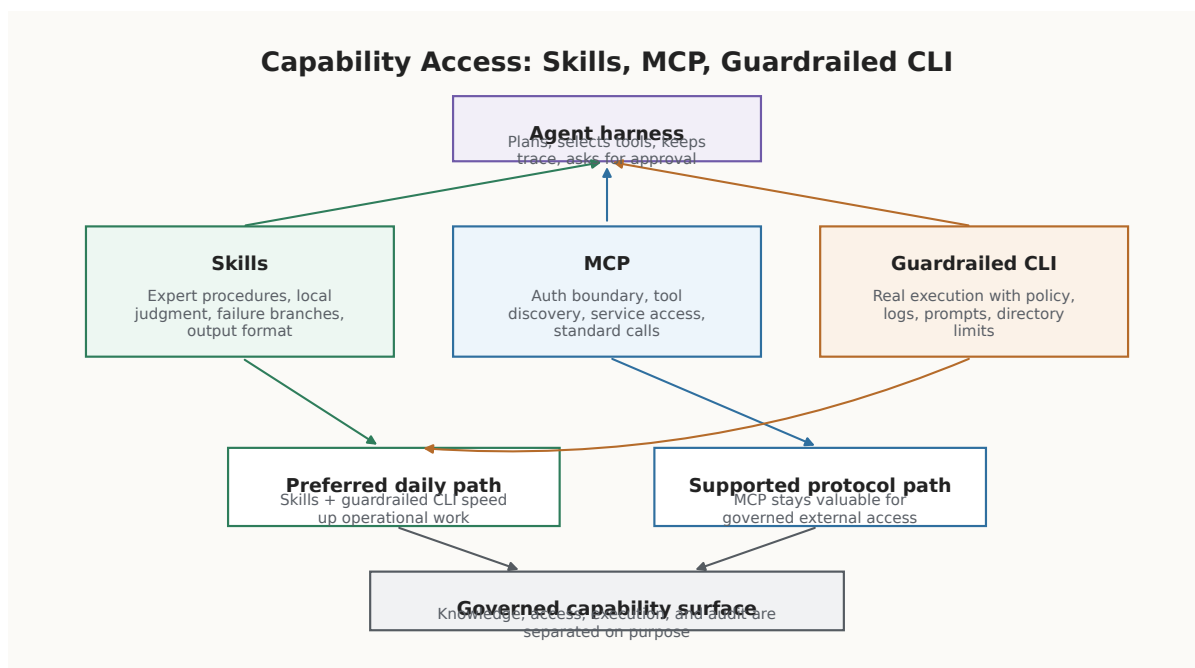


图 15: 根据 00:17:45–00:19:12 关于 Skills、MCP 与 guardrailed CLI 的讨论重绘的说明性示意图。图中把知识封装、协议化工具接入和受控执行分开，强调 KP 偏好的 Skills + guardrailed CLI 路径，同时保留 MCP 在授权和外部工具接入上的价值。

10.2 MCP：认证与外部工具协议的价值

本节定义 MCP。MCP (Model Context Protocol) 是一层用于把模型或智能体连接到外部工具、服务和上下文资源的协议。它的价值不只在“能调工具”，还在把工具访问变得更标准：工具怎样声明能力，调用需要什么参数，访问受什么权限约束，用户或组织怎样授权，返回结果怎样交给模型继续处理。

KP 对 MCP 的态度有明显保留。他认为 MCP 可能带有阶段性热度，但他也明确说，从 auth perspective 看，MCP 很强。这里的 auth 可以理解为授权和认证边界：智能体想访问日历、代码托管、工单、数据库或内部服务时，系统必须知道“谁在请求、能访问什么、凭什么访问、访问结果能否继续传递”。一旦进入企业环境，工具接入的第一难点常常落在权限边界，而非 API 本身。

可以把 MCP 类比成企业大楼里的统一门禁和访客登记系统。没有它，每个房间都要单独判断访客身份，规则分散且难维护；有了它，工具接入可以围绕统一协议建立。这个类比也暴露出 MCP 的局限：门禁系统能决定谁能进门，却不能自动告诉进门的人怎样完成高质量工作。真正的任务方法、判断顺序和组织经验，仍然需要 skills 或其他知识封装来提供。

MCP 的常见误读

MCP 不能自动替代专家工作流。它解决的是工具和服务怎样被模型安全、标准、可治理地访问；它不保证智能体知道何时调用工具、怎样组合多个命令、如何判断结果可信、失败后怎样回退。把 MCP 当成“装上后智能体自然会做事”的万能插座，会低估任务知识和执行护栏的价值。

Ian 随后补充了产品立场：两条路线都会被支持，社区继续使用什么，平台就需要保证这些能力能同 harness 和模型协作。这句话很重要。KP 表达的是个人实践偏好，产品平台承担的是生态兼容责任。对使用者来说，务实选择通常是组合式的：用 MCP 管工具接入和授权，用 skills 管任务知识，用 guardrailed CLI 管可审计执行。

10.3 Guardrailed CLI: 在受控命令行中完成任务

受护栏约束的 CLI 是本节的第三个定义。它指把命令行能力暴露给智能体时，同时设置权限、目录边界、危险命令限制、执行前提示、日志记录、人工审批和结果回传规则。它的核心价值是把真实系统操作变成可观察、可限制、可复盘的步骤。

KP 喜欢 skills 加 guardrailed CLI，背后的原因可以拆成一条执行链。skill 告诉智能体“应该怎么做”；CLI 让智能体“真的能做”；护栏保证“做的过程不越界”。例如排查日志时，skill 可以规定先按时间窗口过滤，再按服务名聚合，再查看错误码分布；CLI 负责实际运行过滤和聚合命令；护栏限制它只能读取指定日志目录，不能修改生产配置，不能把敏感日志随意发送到外部服务。

从平台角度看，这种组合比纯粹的自由工具调用更容易治理。每一次命令都可以留下参数、退出码、标准输出、标准错误和后续解释；每一个 skill 都可以被维护、评估和淘汰；每一类权限都可以被明确分配。智能体因此不再只是“会说怎么做”，也不再是“随便拿到一堆工具”，它被放在一条有知识、有执行、有审计的轨道上。

10.4 本章小结

本节定义了 MCP 和 guardrailed CLI，并把它们同第 4 节已经定义的 skills library 区分开。KP 的实践偏好是 skills 加受护栏 CLI，因为这种组合能把专家经验直接交给智能体，又能让智能体在真实命令行环境中完成排障、测试和生成任务。MCP 的强项在认证和外部工具访问：它让工具接入更标准，让授权边界更清晰。

这三者的关系可以压缩为一句工程判断：skills 管“怎么做”，MCP 管“能安全访问什么”，guardrailed CLI 管“怎样可审计地执行”。平台层面仍会支持多种路线，因为生态会混合使用这些能力；个人 workflow 层面，KP 的经验显示，专家贡献的 skills 与受控 CLI 往往能最快转化为生产效率。

11 Evaluation: 复杂智能体 workflow 怎样被验证

能力接入之后，紧接着要问验证问题。智能体能调用 skill、工具和 CLI，并不自动意味着它能稳定完成任务。KP 对 evaluation 的判断很直白：evaluating this stuff is really hard。难点首先来自机械成本：要为每个问题集启动合适的沙箱环境；随后来自数据成本：要构造新的、贴近目标 skill 或 workflow 的数据集；最后来自元评估：有人已经在尝试让智能体自己设计测试，这会把验证问题推向更高一层。

11.1 沙箱环境搭建的机械成本

本节定义 evaluation。Evaluation 指在受控沙箱条件下，用明确的问题集测试一个智能体 workflow 是否成功的过程。它不同于第 7 节的 observability：observability 负责解释智能体做了什么，evaluation 负责判断它做得是否合格。前者像事故录像和飞行记录仪，后者像驾照考试和道路测试。

复杂智能体 workflow 的评测比普通模型问答更难，因为被测对象不只有模型回答。被测对象包括任务解析、计划生成、工具调用、文件读写、命令执行、浏览器观察、错误恢复、最终报告和人工审批点。只给模型一组选择题，无法覆盖这些环节。平台要搭出一个足够接近真实任务的沙箱：有干净初始状态，有受控文件系统，有可运行命令，有模拟服务或真实服务替身，有预期输出，也有失败时的轨迹记录。

evaluation: 评测对象要包含 workflow

对智能体平台来说，evaluation 不能只问“模型最后答对了吗”。它还要问：环境是否正确初始化，智能体是否选择了合适工具，是否在允许范围内执行，是否记录了关键中间状态，是否避免无效循环，是否把结果整理成可审查产物。复杂 workflow 的质量来自全链路，不只来自最后一句回答。

KP 说“spinning up all of these sandboxed environments”本身就很机械。这里的“机械”并不轻松，含义更接近繁琐且容易出错。每个问题集可能需要不同依赖、不同权限、不同初始文件、不同数据样本和不同验收脚本。若环境搭错，评测结果会失真：智能体失败可能因为题目难，也可能因为依赖缺失；智能体成功也可能只是撞上了泄漏答案或过宽权限。

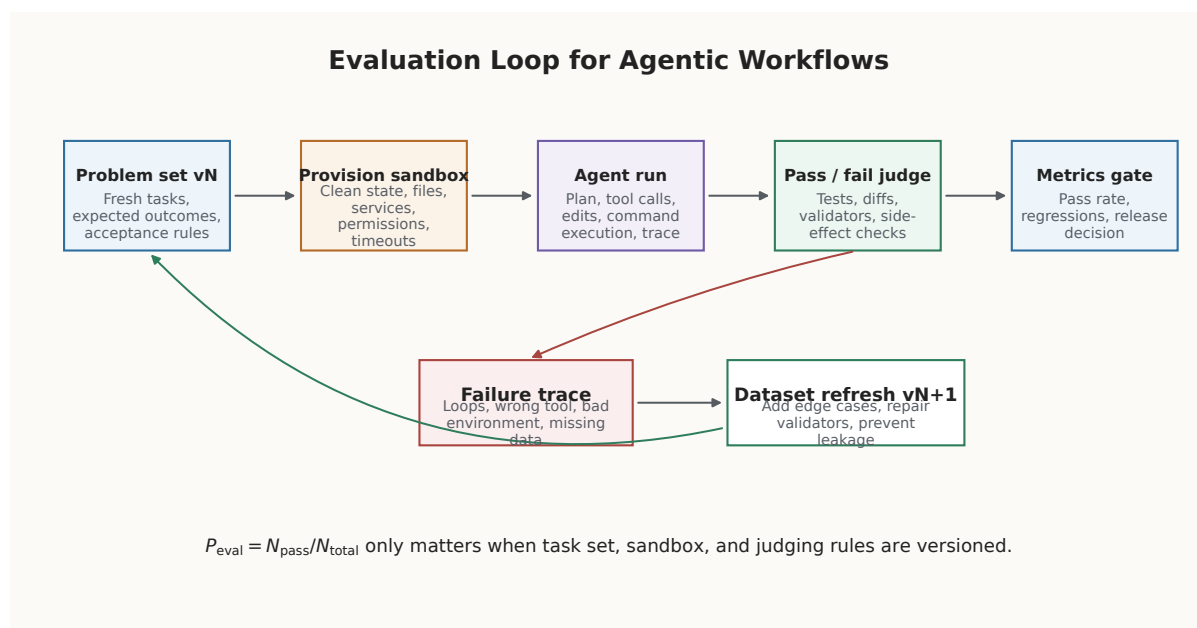


图 16: 根据 00:19:12–00:20:25 关于 agentic workflow evaluation 的讨论重绘的说明性示意图。评测链路从问题集和沙箱初始化开始，经由 agent run 与 pass/fail 判定，再把失败轨迹反馈到数据集刷新。

11.2 新数据集为什么最难

KP 认为最棘手的是 coming up with new datasets。开源 benchmark 对外部比较很有用，但企业内部 skill 和产品 workflow 常常具有强烈的本地性。一个日志排查 skill 的测试集需要真实日志结构的代表性样本；一个代码库迁移 skill 的测试集需要旧 API、新 API、边界情况和回滚场景；一个浏览器自动化 skill 的测试集需要页面状态、DOM 变化和异常弹窗。

新数据集难在三个层面。第一，代表性难。数据集要覆盖真实任务中的常见路径、长尾错误和容易误判的边界。第二，泄漏控制难。如果测试题来自训练材料、文档示例或过去公开答案，评测就会高估能力。第三，验收标准难。智能体任务通常有多种可接受解法，评测不能只比较字符串，还要判断行为、文件差异、测试结果和副作用。

为了说明评测通过率，可以引入一个简单教学公式：

$$P_{eval} = \frac{N_{pass}}{N_{total}}$$

- P_{eval} : evaluation pass rate, 即评测通过率, 表示被测工作流在给定问题集上的通过比例。
- N_{pass} : 通过的 benchmark、sandbox task 或内部问题数量。通过必须按事先定义的验收标准计算。
- N_{total} : 总评测任务数量, 包含通过、失败、超时、环境错误后被计入失败的任务。

这个公式很朴素, 却能逼迫团队说清楚三件事: 什么算 pass, 哪些任务进入分母, 环境错误是否计入失败。若 N_{total} 只包含容易题, P_{eval} 会虚高; 若 N_{pass} 只看最终文本, 不看副作用, 评测也会虚高。真正有用的评测数字必须绑定问题集版本、沙箱配置和验收脚本。

11.3 技能作者、代理生成测试与元评测

KP 提到, 对于 specific skills, 测试责任几乎会落到 skill 作者身上。这很合理。写 skill 的人最清楚该任务的真实边界、常见失败点和验收标准。平台团队可以提供评测框架, 但无法替每个领域设计高质量题目。一个安全扫描 skill、一个日志诊断 skill、一个前端修复 skill、一个文档生成 skill, 成功条件完全不同。

更有意思的是, 现场提到有人实验让 agents designing tests。这个方向很 “meta”, 因为被评估的对象和设计评测的对象都可能是智能体。它的优势是扩展速度快: 智能体可以根据 skill 文档生成更多边界样例、反例和验收脚本。风险也明显: 如果测试生成智能体和解题智能体共享同样盲点, 评测会变成同类模型互相放行。

代理设计测试的双刃剑

让 agent 设计测试可以加快数据集扩展, 尤其适合生成边界案例、变体任务和验收脚本草稿。但这种做法必须有人类或独立评估机制复核。若测试生成者和被测 workflow 依赖同一类模型偏好, 测试可能只覆盖模型擅长的题型, 绕开真实生产里的脏数据、权限问题、环境漂移和副作用。

因此, 成熟评测体系通常需要分层: skill 作者提供领域题目和验收标准; 平台提供统一沙箱、运行记录和统计口径; 智能体可以辅助扩充测试, 但不能独占裁判权; 最终还要用真实失败案例不断回填数据集。这样评测才会从一次性考试, 变成工作流持续改进的闭环。

11.4 本章小结

本节定义了 evaluation: 在受控沙箱条件下, 用明确问题集判断智能体 workflow 是否成功。它的难点来自三块成本: 沙箱环境搭建复杂, 新数据集构造困难, specific skills 的验收标准高度依赖领域知识。公式 $P_{\text{eval}} = N_{\text{pass}}/N_{\text{total}}$ 提醒读者, 评测数字只有在分子、分母、通过标准和环境版本清楚时才有意义。

KP 的回答暴露出一个现实结论: 复杂智能体系统不能只靠 demo 证明能力。Demo 展示的是可能性, evaluation 才能决定能否扩大使用。skill 作者、平台团队、智能体测试生成器和人工复核需要共同组成评测闭环, 否则能力接入越多, 未验证路径也会越多。

12 Quota Management: 模型限制不能中断任务流

评测回答的是 “能不能信任工作流”。进入生产后, 还要回答另一个更用户化的问题: 当模型限制、账户层级和配额耗尽发生时, 任务流会不会断掉。现场问题从 model usage limits 切入, Ian 的回答非

常具体：他们内部账号有时限制更差，因为客户优先；不同层级总会有 limits；未来更常见的模式是某个模型容量或 credit 用完后，在 harness 下无缝转到其他模型，并允许用户给出 preference。

12.1 企业账户、个人账户与不同层级限制

本节定义 quota management。Quota management 指围绕模型和计算预算进行分配、耗尽、降级、恢复和用户提示的策略与运行时行为，目标是在资源有限时仍尽量保持任务连续。第 5 节已经定义 token quota 的经济学，本节聚焦配额耗尽时的产品行为。

Ian 举了一个有趣但很实在的例子：企业内部账号和个人账号的限制可能不同，corporate account 有某些能力，personal account 没有；Google 内部使用者甚至可能因为“客户优先”而遇到比外部用户更差的限制。这说明模型限制并非单一开关，它由账户类型、订阅层级、模型层级、容量状态和组织政策共同决定。

quota management：限制是分层系统

模型限制通常并非一个总闸门。它更像多层水阀：账户层有企业和个人差异，订阅层有普通和高级差异，模型层有 Pro、Flash、本地模型等差异，容量层还会受实时供给影响。Quota management 的任务，是在这些阀门变化时，让工作流知道还能用什么、该怎样降级、何时通知用户。

这也是为什么“用户要改变行为”。当个人账户没有 Ultra 或某个高级模型额度时，用户不能假定所有任务都能持续使用最强模型。平台若只在失败时告诉用户“限额已到”，体验会很差；更好的做法是把限制提前纳入任务计划，让模型选择、预算消耗和 fallback 都成为可预期状态。

12.2 从强模型降级到 Flash 或本地模型

Ian 描述的未来模式是：用户在一个模型上用完 credit 或 capacity 后，系统自动转到其他模型，例如从 Pro 转到 Flash，或者在订阅内所有云端额度耗尽后转到 local model。关键短语是 seamlessly under the harness。也就是说，降级不应该要求用户手动重开任务、复制上下文、重新解释目标。

这里可以用一个强模型依赖比例来表达生产约束：

$$F = \frac{Q_{\text{strong}}}{Q_{\text{total}}}$$

- F ：strong-model dependency fraction，即工作流对强模型配额的依赖比例。
- Q_{strong} ：强模型或高级模型消耗的 quota，例如 Pro、Ultra 或其他高成本模型的 token 与调用预算。
- Q_{total} ：该工作流消耗的总模型 quota，包含强模型、轻量模型、本地模型或其他可计量模型资源。

若 F 接近 1，工作流几乎完全依赖强模型，遇到强模型限额时很容易停摆。若 F 较低，说明大量步骤可由 Flash、本地模型或较便宜模型承担，强模型只用于关键判断点。这个公式不承担精确计费功能，它提醒架构师：要把强模型用在瓶颈判断上，把可降级步骤设计成可替换。

quota fallback: 降级阶梯

一个实用的 fallback 阶梯可以按“强模型 → 快速轻量模型 → 本地模型 → 等待或请求用户确认”组织。每一级都要说明质量影响、速度影响和可执行任务类型。平台不能只在失败后报错，它应当在任务开始前或运行中根据用户 preference、剩余额度和任务风险选择合适层级。

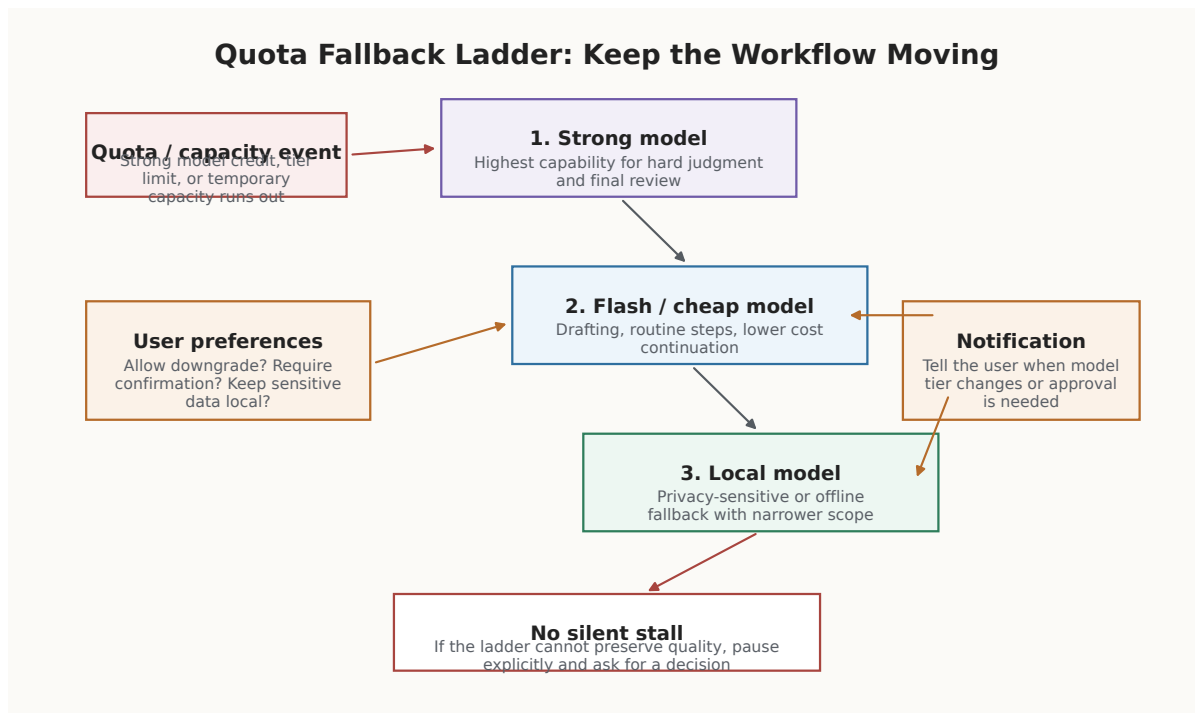


图 17: 根据 00:20:25–00:22:48 关于 model usage limits 与 quota management 的讨论重绘的说明性示意图。阶梯图强调强模型、Flash/低成本模型与本地模型之间的连续降级，并把用户 preference 与 notification 作为避免 silent stall 的控制面。

preference settings 在这里很关键。不同用户对降级的容忍度不同：有人愿意用 Flash 完成草稿，再用强模型复核；有人要求所有代码修改必须经过强模型；有人只允许本地模型处理敏感材料。把 preference 明确化，系统才能在额度耗尽时做出符合用户意图的选择，避免把所有任务一律降级或一律停住。

12.3 把配额耗尽变成可预期状态

Ian 明确说，糟糕体验是用户把任务交给智能体后离开，回来发现它过去一小时什么都没做，因为刚开始就撞上限额。这个风险可以称为 silent stall：任务没有成功，也没有及时失败；它在用户看不到的地方静默停住，消耗的是用户等待时间和信任。

silent stall 对智能体产品尤其危险。普通聊天失败通常发生在用户眼前；长任务智能体的失败可能发生在后台。用户以为系统正在跑测试、读文件、调用浏览器或生成报告，实际只是卡在配额提示、模型排队或重试循环里。若平台不能把状态显式暴露出来，用户会把“等待”误认为“进展”。

配额耗尽的失败模式

配额耗尽最坏的形态是后台任务静默停住。用户离开一小时后回来，看到的是一条迟到的限额提示，完成结果仍然缺席。可规模化的 quota management 必须把“额度不足、正在降级、等待用

户确认、已切换模型、任务暂停”这些状态做成清晰事件，并写入轨迹。

因此，quota management 不只是资源分配，也是一种 UX 诚实。系统需要提前估算任务可能消耗，运行中报告剩余额度，在触发阈值前说明可选降级路径，并在需要人工确认时立即通知。这样用户才能知道任务正在推进、降级执行、等待授权，还是已经无法继续。

12.4 本章小结

本节定义了 quota management：它负责在账户层级、模型层级、容量状态和用户 preference 之间管理预算连续性。Ian 的回答说明，模型限制会长期存在，而且会因企业账号、个人账号和订阅层级而变化。平台需要把“用完某个模型额度”变成可处理状态，避免任务直接中断。

公式 $F = Q_{\text{strong}}/Q_{\text{total}}$ 提醒读者检查工作流对强模型的依赖比例。若所有步骤都绑在强模型上，任何限额都会变成停摆点；若 workflow 提前设计了 Pro 到 Flash 到本地模型的 fallback ladder，并尊重用户 preference，配额耗尽就能从灾难变成可预期降级。真正要避免的是 silent stall：智能体后台卡住一小时，用户却以为任务仍在推进。

13 Automated Code Review：当代理写更多代码，审查也要升级

最后一个问题把讨论落到代码审查。随着智能体生成更多代码，审查环节会被放大。KP 的回答显示，Google 已经有相当多基础设施：按语言划分的 auto review model，会用 style guides 和高质量代码样例微调；按 PA 或产品线配置自己的 SRI、prompt 和信号；PR 中甚至可能出现别人启动的 agent 自动评论，给出相当好的建议。

13.1 按语言微调的自动审查模型

本节定义 code review。Code review 指对代码变更进行人工与自动审查的过程，包含语言风格、代码库约定、测试覆盖、设计风险、安全风险、可维护性和产品线特定要求。智能体写代码后，code review 的任务没有减少，压力反而上升，因为提交规模和提交频率都会增长。

KP 提到 per language basis 的 auto review model。这是一个务实选择。不同语言的风格指南、常见 bug、测试习惯和工程惯例差异很大。C++、Java、Python、Go、TypeScript 的审查重点不相同；同样是“可读性”，在不同语言里对应的结构和禁忌也不同。按语言微调自动审查模型，相当于让审查员先学会每门语言的语法、惯用法和组织规范。

code review：自动审查的第一层

按语言微调的 auto review model 负责提供基础质量信号：代码是否符合语言风格指南，是否违背常见惯用法，是否接近过去被认为优秀的代码样例，是否包含明显测试缺口或低级错误。它不能取代所有人工判断，但能在大规模提交中提前筛出常见问题。

这里的“fine-tuned on style guides and previous good examples of code”很关键。风格指南提供显性规则，例如命名、格式、错误处理、测试结构；优质历史样例提供隐性分布，例如什么样的抽象在本代码库里更自然，什么样的错误处理更符合团队习惯。自动审查模型若只看通用开源代码，很容易给出同项目传统冲突的建议；加入内部样例后，它才更接近组织内部审查员的眼光。

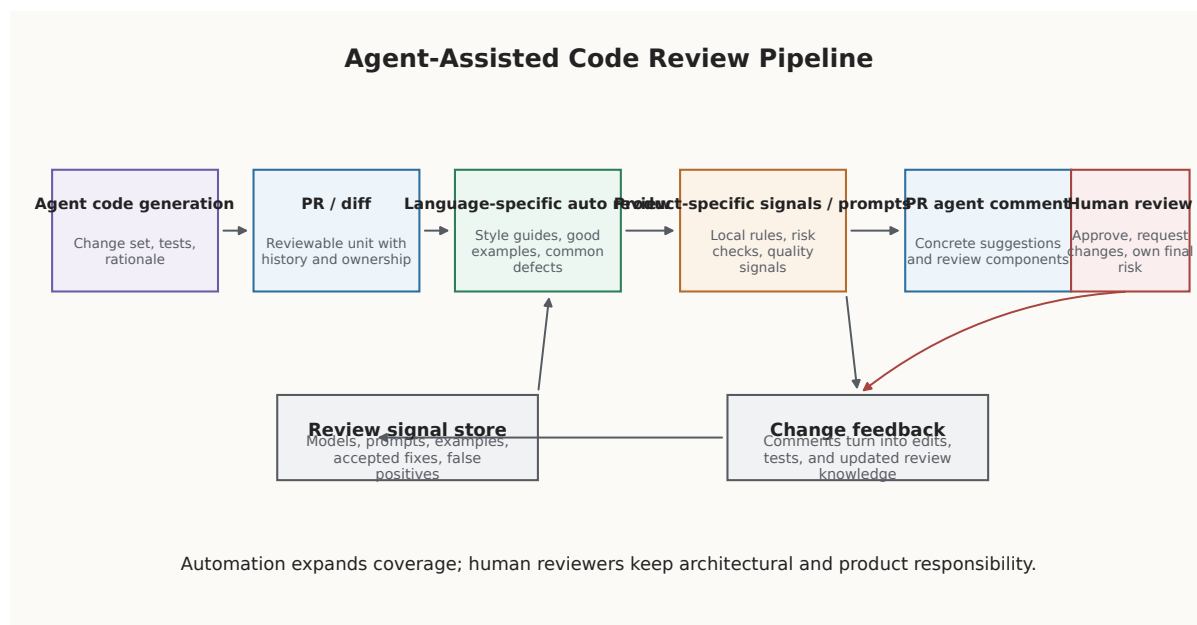


图 18: 根据 00:22:48–00:25:12 关于 automated code review 的讨论重绘的说明性示意图。流水线展示 agent code generation、按语言自动审查、产品线信号与提示、PR agent comment 和 human review 的协作关系，避免展开字幕中不确定的 PA、SRI、Jewels/Jules 缩写。

13.2 按产品线配置的 SRI 与 Prompt

语言只是第一层。KP 接着说，在 PA 或 product basis 上，团队会提供自己的 SRIs、prompts 等，让其他 reviewer 获得更好的 code quality signal。字幕里的“PA”可能是产品区域、产品线或内部组织缩写；这里不强行展开。更稳妥的理解是：代码审查还要按产品上下文定制。

SRI 在字幕中没有展开全称，需要后续复核；从上下文看，它指向产品线用来指导审查信号的内部规则、要求或说明。无论缩写具体含义是什么，机制都清楚：同一段代码在不同产品里风险不同。支付、广告、研究平台、用户界面、基础设施、隐私相关服务，各自需要不同提示词、不同检查项和不同严重性判断。

字幕复核点：SRI 与 PA

本段 ASR 记录为“on a PA or product basis”和“specific SRIs and prompts”。PA 与 SRI 很可能是 Google 内部缩写或现场口误转写，字幕没有给出全称。正文只使用机制层面的解释：按产品线配置额外审查规则、提示和质量信号。最终集成稿若能回看视频或补充讲者资料，应复核这两个缩写。

产品线定制审查像医院里的专科会诊。通用审查模型可以发现发烧、出血、骨折这类明显信号；专科规则知道这个病区最怕哪类并发症。对代码来说，产品线规则会提醒审查员注意特定数据流、特定 API、特定用户承诺、特定可用性要求和特定迁移风险。智能体生成代码越多，这些局部规则越不能只靠人工记忆。

13.3 PR 里的代理评论与审查规模膨胀

KP 还给了一个很具体的例子：他前一天发了一个 PR，甚至没有主动触发 auto review，就有某个别人启动的 agent 在 PR 上评论，并给出不错的建议。这说明审查智能体已经从“被调用的工具”走向

“PR 流程中的参与者”。它可以在审查队列里读取 diff、结合语言模型和产品线提示，生成评论、建议修改或提示风险。

Ian 随后把问题放大到 Google 工程规模：如果 Google 工程师已经提交大量代码，而智能体又开始提交更多代码，审查压力会随代码量一起膨胀。他还提到一个字幕记录为“Jewels”的 Web/GitHub PR 工具；结合语境，它疑似指 Google 的 Jules，但仅凭字幕无法最终确认。正文因此只保留谨慎表述：这类 PR Web 工具可以承载 review components，让代码审查能力进入更熟悉的 PR 工作流。

code review pipeline：从提交到信号

一个可扩展的智能体代码审查流水线通常包含：开发者或编码 agent 提交 PR；语言级 auto review model 检查通用风格和常见问题；产品线 prompts/SRI 注入局部规则；PR agent 生成具体评论；人类 reviewer 判断风险、取舍建议并批准或要求修改。自动审查负责扩展覆盖面，人类审查负责最终责任。

这里也回扣了第 9 节的“digital assembly line supervisor”类比。当智能体开始写更多代码，人类工程师的角色会从逐行生产者转向流水线监督者、设计者和最终责任人。但监督者也需要工具。若代码生成速度上升，审查仍停留在纯人工模式，瓶颈会从“写不完”转移到“看不完”。自动审查的目标是把高频、重复、语言级和产品线级信号提前浮出来，让人类把注意力放在架构、风险和业务语义上。

13.4 本章小结

本节定义了 code review，并解释了为什么智能体生成代码会迫使审查体系升级。Google 的做法包含多层信号：按语言微调的 auto review model 学习 style guides 和优质代码样例；按 PA 或 product basis 配置 SRI、prompt 等局部质量信号，但 PA 与 SRI 的全称无法从字幕确认；PR agent 可以直接在 review 流程里评论，给出具体建议；字幕记录为 Jewels、语境疑似 Jules 的 Web/GitHub PR 工具，则说明 review components 正在进入更熟悉的 PR 工作流。

核心结论很现实：当 agent 让代码产量膨胀，审查也必须获得自动化增幅。否则平台只是把瓶颈从编码迁移到 review。可持续的代码审查体系要把语言级规则、产品线规则、PR 代理评论和人类最终判断组合起来，让智能体既能写更多代码，也能接受更密集、更结构化、更贴近产品上下文的审查。

14 总结与延展：智能体平台的五条控制线

14.1 全片主线压缩：从演示闭环到生产控制

这场分享的价值不在某一个单点功能，而在于它把智能体系统的生产形态摊开给观众看：一个智能体平台要把目标、计划、工具、浏览器或 IDE 状态、轨迹、配额、评测和审查接成闭环。Antigravity 的现场演示提供了最直观的起点：用户提交 spec，模型生成计划，平台调用内部工具，浏览器被打开和控制，DOM 被检查，scratchpad 留下中间轨迹，implementation plan 交给人类确认。下半场 Q&A 则把这个闭环放大到 Google/DeepMind 的组织尺度：当使用者、任务、技能、模型和计算资源一起增长，真正困难的部分会集中到控制线本身。

五条控制线

一套可扩展的智能体平台至少需要同时设计五条控制线：**能力面**决定智能体能调用什么；**上下文经济**决定信息怎样流动和压缩；**配额经济**决定成本和资源怎样被分摊；**可观测性**决定故障怎样被定位；**评测与审查**决定结果怎样进入生产信任链。缺少任意一条，平台都会从“能跑一个 demo”退回到“难以稳定运营”。

这五条控制线可以沿着一个执行循环来理解：

Plan → Act → Observe → Trace → Evaluate → Review → Budget

其中：

- Plan 表示把用户目标转成可执行步骤。
- Act 表示通过文件系统、浏览器、CLI、MCP 或技能调用改变外部状态。
- Observe 表示读取 DOM、运行结果、模型返回值、工具输出或环境状态。
- Trace 表示把步骤、调用、观察和失败点记录到 trajectory store。
- Evaluate 表示在沙盒或任务集上判断工作流是否成功。
- Review 表示让自动审查和人类审查共同检查产物。
- Budget 表示把 token、模型层级、TPU/GPU 资源和用户配额纳入运行时决策。

下面的英文动词只用于压缩前文机制，方便读者把各章节放回同一条执行链，并不新增术语定义。

这个循环解释了为什么演示里一个看似普通的 Web 小游戏任务值得展开讲。它展示的核心结论远超过“模型会写代码”：任务从 spec 到代码、从代码到浏览器、从浏览器到 DOM、从 DOM 到报告、从报告到人类确认，形成了一条完整链路。Q&A 里关于 Deep Research、skills、quota、observability、evaluation、code review 的讨论，实际都在补齐同一个链路的不同环节。

14.2 五类失败模式与对应控制

如果把全片内容转成工程检查表，可以得到下面这张表。它比单纯总结概念更有用，因为智能体平台的风险通常不会以抽象名词出现，而会以某个运行时症状出现：配额突然耗尽、任务卡住、技能太多无人维护、审查信号变弱、研究管线搬运大量上下文。

失败模式	表面症状	控制手段
Agent sprawl	技能、代理和工具包装器不断增加，用户不知道该用哪一个，质量也难以追踪。	建立技能库治理：所有权、版本、淘汰机制、专家贡献、测试用例和可发现性一起管理。
Context blow-up	Deep Research 这类长任务在管线间传递巨大文本块，模型上下文被中间材料占满。	把管线改造成共享工作空间协作：文件、草稿、证据和中间产物留在 workspace，模型只读取当前步骤真正需要的部分。
Quota exhaustion	一个用户启动多个长任务，强模型和长上下文迅速消耗 token 或 TPU/GPU 预算。	按用户和团队统计消耗，设置可预期的限额、降级路径和通知机制，让任务从强模型平滑迁移到便宜模型或本地模型。
Hidden loop	代理在某一步反复尝试，最终用户只看到任务拖慢或失败。	建立 trajectory store 和层级化观测界面，从 workflow 下钻到工具调用、模型请求和具体失败点。
Review overload	代理生成代码量上涨，人工审查被大量 PR 和风格细节淹没。	用语言级自动审查模型、产品线特定信号和 PR 代理评论提供第一层过滤，再由人类审查关键设计和风险。

不要把“自治”理解成免管理

这场分享最容易被误读的地方，是把多智能体、自动审查和工具调用看成越来越少的人类参与。更贴近现场内容的理解是：人类从逐行执行者转为平台主管。主管仍要定义目标、设置边界、审批计划、检查轨迹、分配配额和处理例外。智能体越能行动，控制面越需要清楚。

14.3 对研究、产品和工程团队的具体启示

对研究团队来说，Deep Research 的讨论提示了一个方向：长任务的瓶颈经常来自信息移动方式。把所有搜索结果、草稿和中间判断压进一个上下文窗口，会让模型把注意力浪费在搬运材料上。共享文件系统像研究小组的桌面：资料可以放在同一个地方，不同成员按需读取、补充和标注，最终把上下文窗口留给真正的判断。

对产品团队来说，配额管理和模型降级不应被看成后台账单问题。用户把任务交给智能体后，最糟糕的体验之一是系统静默耗额度，然后停在半路。更成熟的体验应该让用户预先知道成本级别、模型偏好、降级顺序和任务状态。强模型、Flash 类模型、本地模型之间的切换，应当成为 harness 的运行能力，而非用户手动抢救失败任务的补丁。

对工程团队来说，可观测性和评测需要尽早设计。日志只告诉团队发生过一些事件；trajectory store 要回答的是“代理在第几步开始偏离目标，哪个工具输出触发了错误判断，哪个模型请求消耗了异常预算”。评测也类似，单个 benchmark 分数不足以覆盖复杂工作流。沙盒环境、任务集、技能级测试和自动生成测试会逐步变成 agent harness 的基础设施。

一个可操作的架构压缩

沿用前文定义，可以把智能体平台类比成一座小型工厂：skills 提供作业指导，MCP 提供受治理的外部工具接入协议，workspace 承载工作台，context window 类似短期记忆容量，quota 类似资源电表，trajectory store 类似过程记录，evaluation 类似质检台，code review 类似出厂审核。这个类比只用于压缩前文关系，具体定义仍以前文各自负责的章节为准。

14.4 开放问题：当平台继续放大

片尾的现场交流邀请属于会议安排，技术内容已经在前面的 Q&A 中收束。真正值得继续追问的是三类开放问题。

第一，技能治理能不能形成稳定生态。大组织内的专家技能有巨大价值，但技能的生命周期、兼容性、测试和淘汰机制会决定它们能否长期复用。没有治理的技能库会变成杂物间，有治理的技能库才可能成为组织知识的运行时入口。

第二，多智能体协作怎样保持透明。并行轨道能提高吞吐量，也会制造新的不透明性：哪个代理拥有哪段状态，代理之间怎样通信，冲突怎样解决，人类在什么层级介入。未来的 harness 需要把这些拓扑结构显式呈现出来，让人能像管理装配线一样调度和纠偏。

第三，自动审查怎样跟上自动生成。代理写代码会带来更多 PR、更多风格差异和更多边界案例。自动审查模型能承担语言规范、产品约束和常见缺陷的第一层过滤，但最终信任仍来自可追踪的证据链：谁生成了变更，谁检查了变更，哪些测试通过，哪些风险由人类确认。

14.5 本章小结

这份笔记的主线可以压缩为一句话：智能体规模化要求团队同时工程化能力、上下文、配额、轨迹、评测和审查，单纯提高模型能力远远不够。Antigravity 的演示给出了最小闭环；Deep Research、skills、quota、observability、evaluation 和 automated code review 则展示了这个闭环在组织尺度下会遇到的压力。能跑任务只是起点，能解释、能限制、能复核、能降级、能持续改进，才是智能体平台进入真实生产环境的门槛。